

Programação I & II

Apresentação - Quem sou eu?

- Victor Ramos Mello
 - Engenheiro de segurança sênior na Microsoft
 - <https://linktr.ee/m0nad>

Apresentação – quem são vocês

- Se apresentem, falem um pouco de vocês.

O que vamos fazer aqui?

- Programação I e II
 - Python3

Livro texto

- Introdução à programação com Python
 - Nilo Ney Coutinho Menezes

Metodologia

- “Se você quer aprender a nadar, pule na água. Em terra firme, nenhum estado de espírito jamais irá ajudá-lo.”
 - Bruce Lee

O que é programação?

- “Programação é o processo de escrita, teste e manutenção de um programa de computador. O programa é escrito em uma linguagem de programação, embora seja possível, com alguma dificuldade, o escrever diretamente em linguagem de máquina. Diferentes partes de um programa podem ser escritas em diferentes linguagens.”
 - https://pt.wikipedia.org/wiki/Programa%C3%A7%C3%A3o_de_computadores

Usando o interpretador

- Barra de pesquisar: “idle”
- Clique no ‘IDLE (Python 3.11 ...’

O que é programar?

- É mandar o computador fazer o que você quer!

O que é uma linguagem de programação?

- “A linguagem de programação é um método padronizado, formado por um conjunto de regras sintáticas e semânticas, de implementação de um código fonte - que pode ser compilado e transformado em um programa de computador,[1] ou usado como script interpretado - que informará instruções de processamento ao computador.”
 - https://pt.wikipedia.org/wiki/Linguagem_de_programa%C3%A7%C3%A3o

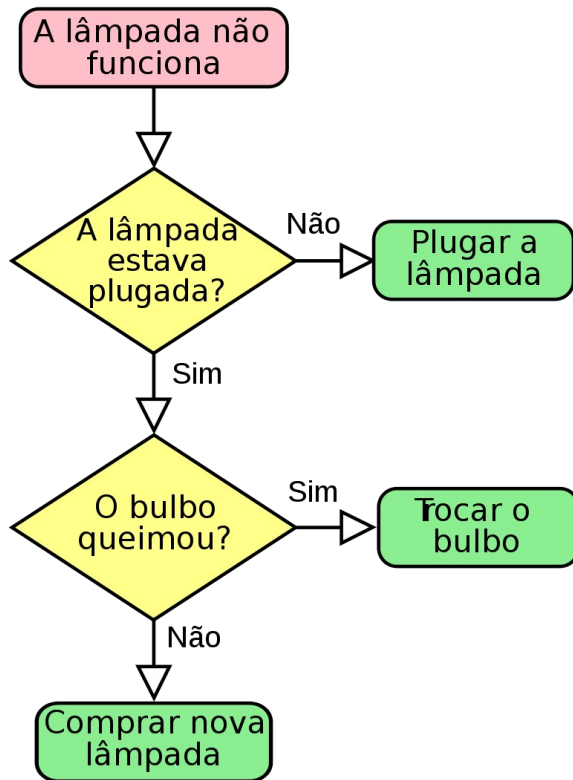
O que é um algoritmo?

- “Em matemática e ciência da computação, um algoritmo é uma sequência finita de ações executáveis que visam obter uma solução para um determinado tipo de problema.”
 - <https://pt.wikipedia.org/wiki/Algoritmo>

O que é um algoritmo?

- É a “receita de bolo” para dizer ao computador o que fazer.

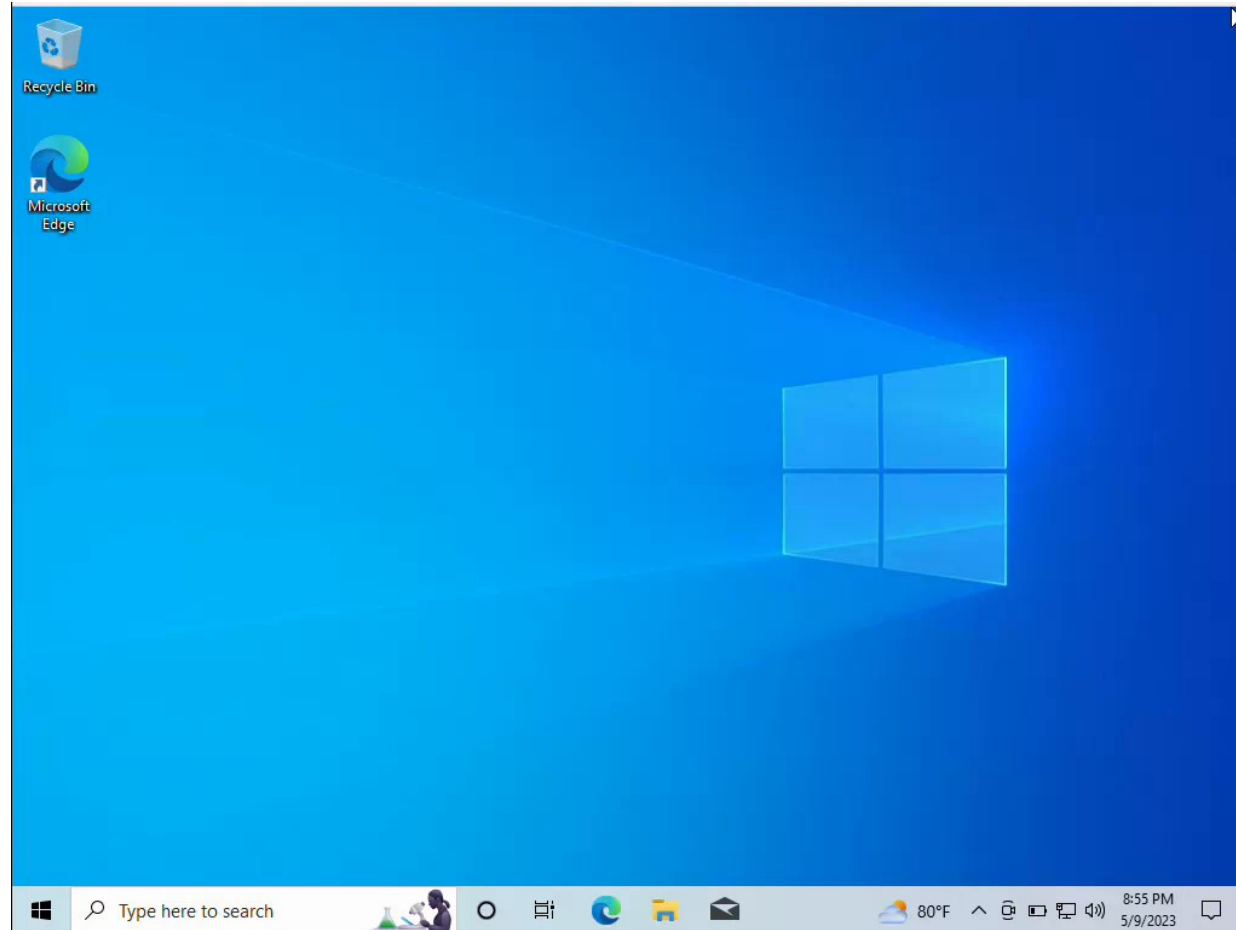
O que é um algoritmo?



Preparando o ambiente

- Instalando Python3 no Windows
- Instalando Python3 no Linux
 - Obs: Se você utiliza Linux(ou MacOS), provavelmente já está instalado.

Windows



Debian/Ubuntu

- `sudo apt install python3`
- `sudo apt install idle-python3`

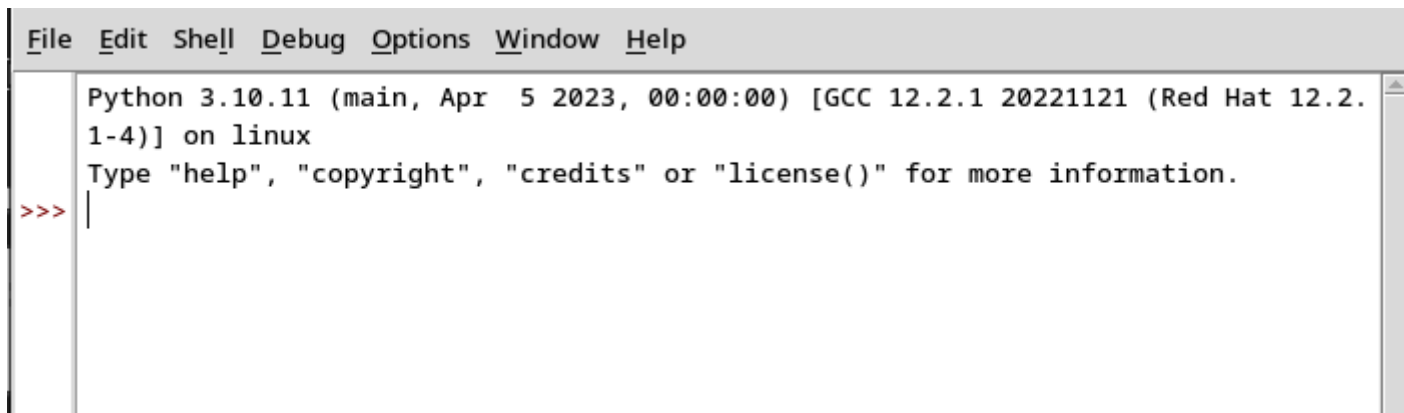
Redhat/Fedora

- `sudo dnf install python3`
- `sudo dnf install python3-idle`

Usando o interpretador

- Linux
 - `$ idle3.10 &`

Usando o interpretador

A screenshot of a Python interpreter window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the Python version and environment information: 'Python 3.10.11 (main, Apr 5 2023, 00:00:00) [GCC 12.2.1 20221121 (Red Hat 12.2.1-4)] on linux'. Below this, it says 'Type "help", "copyright", "credits" or "license()" for more information.' The prompt '>>>' is visible on the left side of the text area, followed by a vertical cursor line.

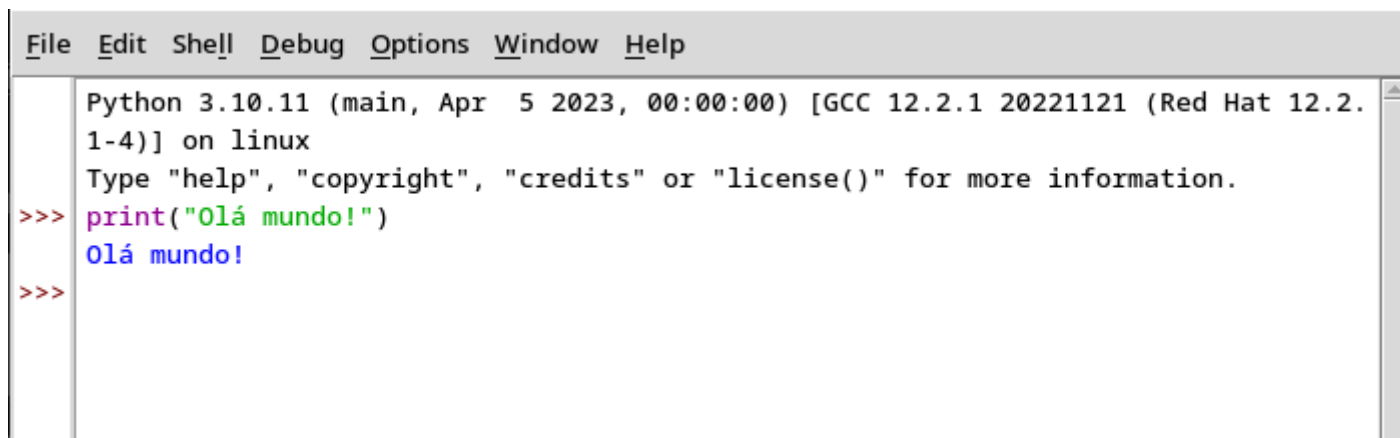
```
File Edit Shell Debug Options Window Help
Python 3.10.11 (main, Apr 5 2023, 00:00:00) [GCC 12.2.1 20221121 (Red Hat 12.2.1-4)] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

Usando o interpretador

- Digite:

```
print("Olá mundo!") <enter>
```

Usando o interpretador



```
File Edit Shell Debug Options Window Help
Python 3.10.11 (main, Apr 5 2023, 00:00:00) [GCC 12.2.1 20221121 (Red Hat 12.2.1-4)] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Olá mundo!")
Olá mundo!
>>>
```

Demo

- Usando o editor

Cuidado ao escrever programas

- Letras maiúsculas e minúsculas são diferentes.
- Aspas são muito importantes e não devem ser esquecidas.
- Parênteses não são opcionais em Python.
- Espaços são muito importantes.

Os primeiros programas

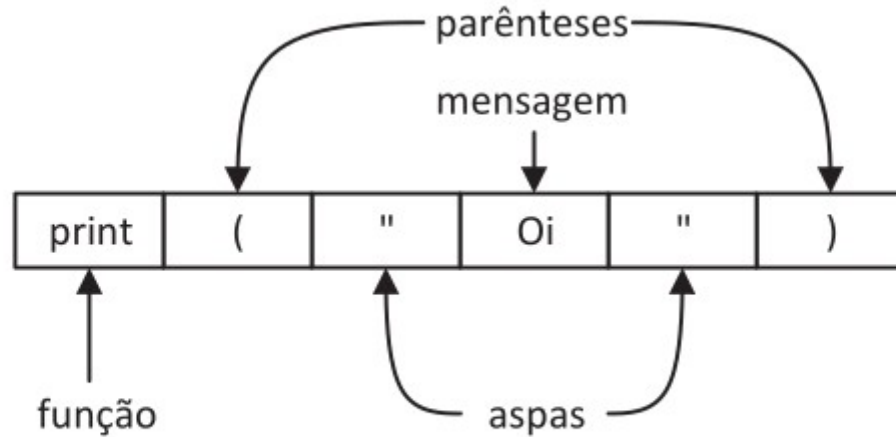


Figura 2.21 – Nomes dos elementos do primeiro programa.

Os primeiros programas

- A função print informa que vamos exibir algo na tela.
 - Utilizamos aspas para indicar o início e o fim do texto de nossa mensagem.
 - A mensagem é nosso texto em si e será exibida exatamente como digitada.
 - Os parênteses são utilizados para separar os parâmetros de uma função, no caso os de print.
 - Um parâmetro é um valor passado para uma função: no caso da função print, a mensagem a imprimir.

Soma e subtração

```
>>> 2+3
```

5

```
>>> 5-3
```

2

```
>>> 10-4+2
```

8

Multiplicação e divisão

```
>>> 2*10
```

```
20
```

```
>>> 20/4
```

```
5.0
```

Exponenciação

```
>>> 2**3
```

```
8
```

Resto da divisão inteira(módulo)

```
>>> 10 % 3
```

```
1
```

```
>>> 16 % 7
```

```
2
```

```
>>> 63 % 8
```

```
7
```

Parênteses

Os parênteses são utilizados em Python da mesma forma que em expressões matemáticas, ou seja, para alterar a ordem de execução de uma operação.

```
>>> 3*(2+2)
```

```
12
```

Parênteses

A expressão $1500 + (1500 * 5 / 100)$ é equivalente a:

$$1500 + \left(\frac{1500 \times 5}{100} \right)$$

Obs: Não se esqueça de que, tanto em Python como na matemática, as operações de mesma prioridade são realizadas da esquerda para direita.

Exercício 2.1

- Converta as seguintes expressões matemáticas para que possam ser calculadas usando o interpretador Python.

$$10 + 20 \times 30$$

$$4^2 \div 30$$

$$(9^4 + 2) \times 6 - 1$$

Variáveis

“Na programação, uma variável é um objeto (uma posição, frequentemente localizada na memória) capaz de reter e representar um valor ou expressão.”

- [https://pt.wikipedia.org/wiki/Vari%C3%A1vel_\(programa%C3%A7%C3%A3o\)](https://pt.wikipedia.org/wiki/Vari%C3%A1vel_(programa%C3%A7%C3%A3o))

Variáveis

Em programação, variáveis são utilizadas para armazenar valores e para dar nome a uma área de memória do computador onde armazenamos dados.

Variáveis

Para armazenar algo nesses compartimentos, usaremos o símbolo de igualdade (=) entre o nome do compartimento e o valor que queremos armazenar.

Variáveis - Exemplo:

```
>>> x = 5
```

```
>>> y = 3
```

```
>>> print(x*y)
```

```
15
```

Variáveis

Você pode estar-se perguntando por que criamos duas variáveis, x e y, para multiplicar dois números? Ora, poderíamos fazer assim:

```
>>> print(3*5)
```

15

Variáveis

Ou mesmo assim:

```
>>> print(15)
```

```
15
```

Variáveis

Então, por que escolhemos resolver o problema usando variáveis?

Variáveis

Então, por que escolhemos resolver o problema usando variáveis?

- Claridade do programa
- Reutilização de variáveis e do código
- Armazenar entradas de dados do usuário
- Armazenar resultados de cálculos
- Etc.

Variáveis – Uso prático

```
>>> salario = 2000
```

```
>>> aumento = 5/100
```

```
>>> print(salario + (salario * aumento))
```

```
2100.0
```

Variáveis – Uso prático

O programa da listagem 2.11 pode ser escrito de forma mais direta, utilizando outra fórmula sem variáveis:

```
>>> print(2000 + (2000 * 5 / 100))
```

```
2100.0
```

Variáveis – Uso prático

O programa da listagem 2.11 pode ser escrito de forma mais direta, utilizando outra fórmula sem variáveis:

```
>>> print(2000 + (2000 * 5 / 100))
```

```
2100.0
```

O resultado é idêntico: a diferença está na clareza da representação de nosso problema.

Variáveis – Uso prático

Se mudarmos o valor do salário, na primeira linha do primeiro programa (com variáveis), obteremos o resultado correto na saída do programa, sem precisar nos preocuparmos novamente com a fórmula do cálculo.

Observe também que, se fizermos a mesma coisa no segundo programa (sem variáveis), teremos que mudar o valor de salário em duas posições diferentes da fórmula, aumentando nossas chances de nos esquecermos de uma delas e, conseqüentemente, de recebermos um resultado incorreto.

Exercícios (2.3 – 2.6)

- 1) Faça um programa que exiba seu nome na tela.
- 2) Escreva um programa que exiba o resultado de $2a \times 3b$, onde “a” vale 3 e “b” vale 5.
- 3) Faça um programa que calcule a soma de três variáveis.
- 4) Faça um programa que calcule um aumento de 15% para um salário de R\$ 1750.

Nomes de variáveis

Nome	Válido	Comentários
a1	Sim	Embora contenha um número, o nome a1 inicia com letra.
velocidade	Sim	Nome formado por letras.
velocidade90	Sim	Nome formado por letras e números, mas iniciado por letra.
salário_médio	Sim	O símbolo sublinha (<code>_</code>) é permitido e facilita a leitura de nomes grandes.
salário médio	Não	Nomes de variáveis não podem conter espaços em branco.
b	Sim	O sublinha (<code></code>) é aceito em nomes de variáveis, mesmo no início.
1a	Não	Nomes de variáveis não podem começar com números.

Tipos de dados

- Python tem vários tipos de dados, mas os mais comuns são números inteiros, números de ponto flutuante, booleanos(chamadas de variáveis de tipo lógico), e strings

Variáveis numéricas

- Dizemos que uma variável é numérica quando armazena números inteiros ou de ponto flutuante.

Variáveis numéricas

Os números inteiros são aqueles sem parte decimal, como 1, 0, -5, 550, -47, 30000.

```
>>> x = -4
```

```
>>> y = 3000
```

```
>>> x
```

```
-4
```

```
>>> y
```

```
3000
```

Variáveis numéricas

Números de ponto flutuante ou decimais são aqueles com parte decimal, como 1.0, 5.478, 10.478, 30000.4

```
>>> x = 3.1415
```

```
>>> y = 1.618
```

```
>>> x
```

```
3.1415
```

```
>>> y
```

```
1.618
```

Exercício 3.1

Número	Tipo numérico
5	☐ inteiro ☐ ponto flutuante
5.0	☐ inteiro ☐ ponto flutuante
4.3	☐ inteiro ☐ ponto flutuante
-2	☐ inteiro ☐ ponto flutuante
100	☐ inteiro ☐ ponto flutuante
1.333	☐ inteiro ☐ ponto flutuante

Variáveis booleanas (ou de tipo Lógico)

Armazenar “True”(verdadeiro) ou “False”(falso) em uma variável.

```
>>> verdadeiro = True
```

```
>>> verdadeiro
```

```
True
```

```
>>> falso = False
```

```
>>> falso
```

```
False
```

Comparações lógicas

- Para realizarmos comparações lógicas, utilizaremos operadores relacionais.

Comparações lógicas

Operador	Operação	Símbolo matemático
==	igualdade	=
>	maior que	>
<	menor que	<
!=	diferente	≠
>=	maior ou igual	≥
<=	menor ou igual	≤

Comparações lógicas

```
>>> 1 == 1
True
>>> 1 == 2
False
>>> 1 > 2
False
>>> 1 < 2
True
```

Comparações lógicas

```
>>> 1 != 1
False
>>> 1 != 2
True
>>> 1 >= 2
False
>>> 1 <= 2
True
```


Comparações lógicas

```
>>> x = 1
>>> y = 2
>>> x == y
False
>>> x < y
True
>>> x < 2
True
>>> y == 2
True
>>> y >= 2
True
>>> x <= 2
True
```

Comparações lógicas

Variáveis de tipo lógico também podem ser utilizadas para armazenar o resultado de expressões e comparações.

```
>>> salario_minimo = 1302.00 # Salário mínimo no Brasil em abril de 2023
```

```
>>> salario_minimo_necessario = 6676.11 # Salário mínimo necessário no Brasil em abril de 2023 segundo o DIEESE
```

```
>>> temos_o_minimo = salario_minimo >= salario_minimo_necessario
```

```
>>> print(temos_o_minimo)
```

```
False
```

Exercício 3.2

- Considere $a = 4$, $b = 10$, $c = 5.0$, $d = 1$ e $f = 5$

Expressão	Resultado
$a == c$	☐ True ☐ False
$a < b$	☐ True ☐ False
$d > b$	☐ True ☐ False
$c != f$	☐ True ☐ False
$a == b$	☐ True ☐ False
$c < d$	☐ True ☐ False

$b > a$	☐ True ☐ False
$c >= f$	☐ True ☐ False
$f >= c$	☐ True ☐ False
$c <= c$	☐ True ☐ False
$c <= f$	☐ True ☐ False

Operadores lógicos

- Para agrupar operações com lógica booleana, utilizaremos operadores lógicos.
- Python suporta três operadores básicos: not (não), and (e), or (ou). Esses operadores podem ser traduzidos como “não” ($\neg$ negação), “e” ($\wedge$ conjunção) e “ou” ($\vee$ disjunção).

Operadores lógicos

Operador Python	Operação
<code>not</code>	não
<code>and</code>	e
<code>or</code>	ou

Operadores lógicos

- Cada operador obedece a um conjunto simples de regras, expresso pela tabela verdade desse operador.

A tabela verdade demonstra o resultado de uma operação com um ou dois valores lógicos ou operandos.

Operadores lógicos

- Quando o operador utiliza apenas um operando, dizemos que é um operador unário. Ao utilizar dois operandos, é chamado operador binário. O operador de negação (not) é um operador unário. or (ou) e and (e) são operadores binários, precisando, assim, de dois operandos.

Operador not

V_1	$\text{not } V_1$
V	F
F	V

O operador not (não) é o mais simples, pois precisa apenas de um operador. A operação de negação também é chamada de inversão, pois um valor verdadeiro negado se torna falso e vice-versa.

Operador not

```
>>> not True
```

```
False
```

```
>>> not False
```

```
True
```

```
>>>
```

Operador and

V_1	V_2	$V_1 \text{ and } V_2$
V	V	V
V	F	F
F	V	F
F	F	F

O operador and (e) tem sua tabela verdade representada na tabela 3.5. O operador and (e) resulta verdadeiro apenas quando seus dois operadores forem verdadeiros.

Operator and

```
>>> True and True  
True
```

```
>>> True and False  
False
```

```
>>> False and True  
False
```

```
>>> False and False  
False
```

Operador or

V_1	V_2	$V_1 \text{ or } V_2$
V	V	V
V	F	V
F	V	V
F	F	F

A regra fundamental do operador or (ou) é que ele resulta em falso apenas se seus dois operadores também forem falsos. Se apenas um de seus operadores for verdadeiro, ou se os dois forem, o resultado da operação será verdadeiro.

Operator or

```
>>> True or True
```

```
True
```

```
>>> True or False
```

```
True
```

```
>>> False or True
```

```
True
```

```
>>> False or False
```

```
False
```

Exercício 3.3

- Considere $a = \text{True}$, $b = \text{False}$ e $c = \text{True}$

a and a	☐ True ☐ False
b and b	☐ True ☐ False
not c	☐ True ☐ False
not b	☐ True ☐ False
not a	☐ True ☐ False
a and b	☐ True ☐ False
b and c	☐ True ☐ False

a or c	☐ True ☐ False
b or c	☐ True ☐ False
c or a	☐ True ☐ False
c or b	☐ True ☐ False
c or c	☐ True ☐ False
b or b	☐ True ☐ False

Expressões lógicas

Os operadores lógicos podem ser combinados em expressões lógicas mais complexas. Quando uma expressão tiver mais de um operador lógico, avalia-se o operador not (não) primeiramente, seguido do operador and (e) e, finalmente, or (ou).

Expressões lógicas

True or False and not True

True or False and False

True or False

True

Expressões lógicas

- Os operadores relacionais também podem ser utilizados em expressões com operadores lógicos.

salário > 1000 and idade > 18

Expressões lógicas

Nesses casos, os operadores relacionais devem ser avaliados primeiramente.

salário > 1000 and idade > 18

100 > 1000 and 20 > 18

False and True

False

Exercício 3.4

- Escreva uma expressão para determinar se uma pessoa deve ou não pagar imposto. Considere que pagam imposto pessoas cujo salário é maior que R\$ 2.200,00.

Exercício 3.5

Calcule o resultado: “A > B and C or D”

A	B	C	D	Resultado
1	2	True	False	
10	3	False	False	
5	1	True	True	

Exercício 3.6

Escreva uma expressão que será utilizada para decidir se um aluno foi ou não aprovado. Para ser aprovado, todas as médias do aluno devem ser maiores que 7. Considere que o aluno cursa apenas três matérias, e que a nota de cada uma está armazenada nas seguintes variáveis: matéria1, matéria2 e matéria3.

Variáveis do tipo “string”

- Variáveis do tipo string armazenam cadeias de caracteres como nomes e textos em geral. Chamamos cadeia de caracteres uma sequência de símbolos como letras, números, sinais de pontuação etc.

Strings

- Exemplo: “João e Maria comem pão.”

Nesse caso, “João” é uma sequência com as letras “J”, “o”, “ã”, “o”. Para simplificar o texto, utilizaremos o nome string para mencionar cadeias de caracteres.

Strings

Podemos imaginar uma string como uma sequência de blocos, onde cada letra, número ou espaço em branco ocupa uma posição

String																					
J	o	ã	o		e		M	a	r	i	a		c	o	m	e	m		p	ã	o

Strings

O tamanho de uma string pode ser obtido utilizando-se a função “len()”.

```
>>> len("A")
```

```
1
```

```
>>> len("AB")
```

```
2
```

```
>>> len("hack the planet")
```

```
15
```

```
>>> len("")
```

```
0
```

Strings

Uma string em Python tem um tamanho associado, assim como um conteúdo que pode ser acessado caractere a caractere.

```
>>> str = "João e Maria"
```

```
>>> str[0]
```

```
'J'
```

```
>>> str[3]
```

```
'o'
```

```
>>> str[5]
```

```
'e'
```

Sabendo que uma string tem um determinado tamanho, podemos acessar seus caracteres utilizando um número inteiro para representar sua posição. Esse número é chamado de índice, e começamos a contar de zero. Isso quer dizer que o primeiro caractere da string é de posição ou índice 0.

Strings

String

0	1	2	3	4	5	6	7	8	← Índice
A	B	C	D	E	F	G	H	I	← Conteúdo

Strings

Para acessar os caracteres de uma string, devemos informar o índice ou posição do caractere entre colchetes ([]).

Como o primeiro caractere de uma string é o de índice 0, podemos acessar valores de 0 até o tamanho da string menos 1.

Strings

```
>>> hack = "hack the planet"
```

```
>>> hack[0]
```

```
'h'
```

```
>>> hack[13]
```

```
'e'
```

```
>>> hack[10]
```

```
'l'
```

```
>>> hack[9]
```

```
'p'
```

```
>>> hack[777]
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
IndexError: string index out of range
```

```
>>>
```

Operações com strings

- As variáveis de tipo string suportam operações como concatenação, fatiamento, e composição.

Concatenação

- A concatenação nada mais é que poder juntar duas ou mais strings em uma nova string maior.

Concatenação

- O conteúdo de variáveis string podem ser somados, ou melhor, concatenados.
- Para concatenar duas strings, utilizamos o operador de adição (+). Assim, "AB" + "C" é igual a "ABC". Um caso especial de concatenação é a repetição de uma string várias vezes.
- Para isso, utilizamos o operador de multiplicação (*): "A" * 3 é igual a "AAA".

Concatenação

```
>>> s = "A" + "B"
```

```
>>> print(s)
```

```
AB
```

```
>>> print(s + "C")
```

```
ABC
```

```
>>> print(s + "C" * 3)
```

```
ABCCC
```

```
>>> print("-" * 10 + "X" + "-" * 10)
```

```
-----X-----
```

```
>>>
```

Composição

Juntar várias strings para construir uma mensagem nem sempre é prático. Por exemplo, exibir que "João tem X anos", onde X é uma variável numérica.

Composição

Usando a composição de strings do Python, podemos escrever de forma simples e clara:

```
>>> "João tem %d anos" % X
```

Onde o símbolo de % foi utilizado para indicar a composição da string anterior com o conteúdo da variável X. O %d dentro da primeira string é o que chamamos de marcador de posição. O marcador indica que naquela posição estaremos colocando um valor inteiro.

Composição

Marcador	Tipo
%d	Números inteiros
%s	Strings
%f	Números decimais

Composição

Número ocupando três posições

```
>>> idade = 22
```

```
>>> print("[%d]" % idade)
```

```
[22]
```

```
>>> print("[%03d]" % idade)
```

```
[022]
```

```
>>> print("[%3d]" % idade)
```

```
[ 22]
```

```
>>> print("[% -3d]" % idade)
```

```
[22 ]
```

Composição

Números decimais (pontos flutuantes/floating pointers)

```
>>> print("%5f" % 5)
```

```
5.000000
```

```
>>> print("%5.2f" % 5)
```

```
5.00
```

```
>>> print("%10.5f" % 5)
```

```
5.00000
```

O primeiro indica o tamanho total em caracteres a reservar; e o segundo, o número de casas decimais.

Composição

Exibindo vários valores

```
>>> nome = "João"
```

```
>>> idade = 22
```

```
>>> grana = 51.34
```

```
>>> print("%s tem %d anos e R$%5.2f no bolso." %  
(nome, idade, grana))
```

João tem 22 anos e R\$51.34 no bolso

Extra: print()

```
nome = "João"
```

```
idade = 30
```

```
print("Meu nome é %s e eu tenho %d anos de  
vida." % (nome, idade))
```


Extra: print()

```
nome = "João"
```

```
idade = 30
```

```
print(f"Meu nome é {nome} e eu tenho {idade}  
anos de vida.")
```

Extra: print()

```
nome = "João"
```

```
idade = 30
```

```
print("Meu nome é", nome, "e eu tenho", idade,  
      "anos de vida.", sep=" | ", end="\n\n")
```

Extra: print()

```
nome = "João"
```

```
idade = 30
```

```
print("Meu nome é {} e eu tenho {} anos de  
vida.".format(nome, idade))
```

Extra: print()

```
nome = "João"
```

```
idade = 30
```

```
print("Meu nome é " + nome + " e eu tenho " +  
str(idade) + " anos de vida.")
```

Fatiamento(slices)

```
>>> s = "Do Zero Ao Um"
```

```
>>> s[3:]
```

```
'Zero Ao Um'
```

```
>>> s[:]
```

```
'Do Zero Ao Um'
```

```
>>> s[3:-6]
```

```
'Zero'
```

Podemos também omitir o número da esquerda ou o da direita para representar do início ou até o final.

Podemos também utilizar valores negativos para indicar posições a partir da direita. Assim -1 é o último caractere; -2, o penúltimo; e assim por diante.

Fatiamento(slices)

- `>>> s = "Do Zero Ao Um"`
- `>>> s[0:2]`
- `'Do'`
- `>>> s[8:13]`
- `'Ao Um'`
- `>>> s[3:13]`
- `'Zero Ao Um'`
- `>>> s[3:7]`
- `'Zero'`
- `>>>`

Entenda `[0:2]` como a fatia de caracteres da posição “0” até a posição “2”, sem incluí-la, ou o intervalo fechado em “0” e aberto em “2”.

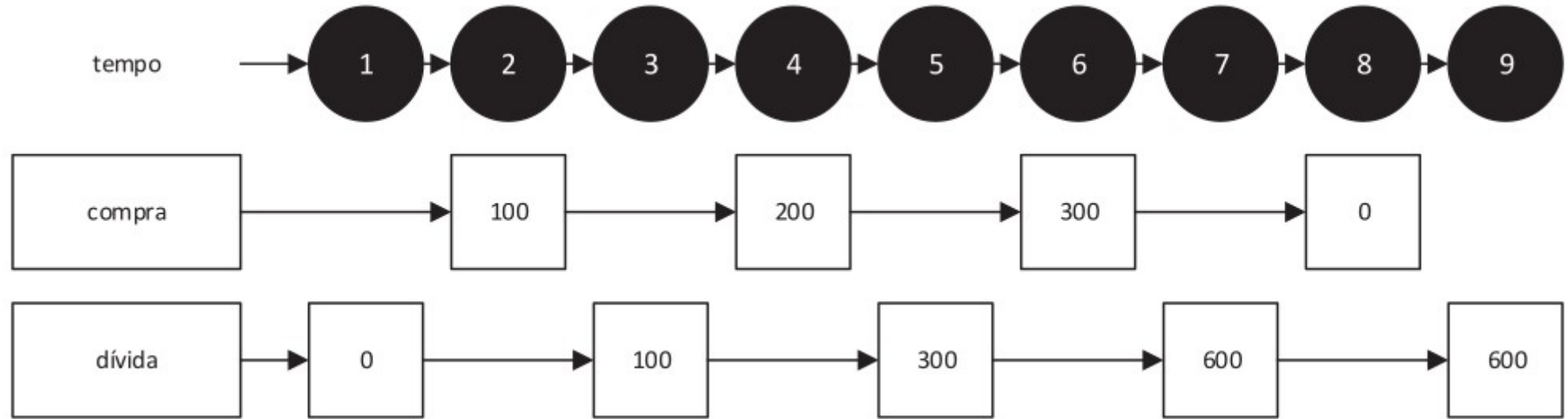
Teste de mesa/Rastreamento

- Um programa é executado linha por linha pelo computador, executando as operações descritas no programa uma após a outra.
- Quando trabalhamos com variáveis, devemos nos lembrar de que o conteúdo de uma variável pode mudar com o tempo.

Teste de mesa/Rastreamento

```
>>> divida = 0
>>> compra = 100
>>> divida = divida + compra
>>> compra = 200
>>> divida = divida + compra
>>> compra = 300
>>> divida = divida + compra
>>> compra = 0
>>> print(divida)
```


Teste de mesa/Rastreamento



Teste de mesa/Rastreamento

- Uma das principais diferenças entre ler um texto e um programa é justamente seguir as mudanças de valores de cada variável conforme o programa é executado.
- Um programa não pode ser lido como um texto, mas cuidadosamente analisado linha a linha.

Teste de mesa/Rastreamento

- O teste geralmente consiste da construção de uma tabela com várias colunas e várias linhas, com cada coluna mostrando uma variável e cada linha mostrando cada entrada de número no algoritmo e os valores subsequentes das variáveis; o processo se dá a partir da simulação da execução do código, instrução a instrução, registrando na tabela cada alteração no valor das variáveis em uma nova linha.
 - https://pt.wikipedia.org/wiki/Teste_de_mesa

Teste de mesa/Rastreamento

- `>>> i = 1`
- `>>> x = i * 2`
- `>>> i = 2`
- `>>> x = i * 2`
- `>>> i = 3`
- `>>> x = i * 2`
- `>>> i = 4`
- `>>> x = i * 2`

Teste de mesa/Rastreamento

i	x
1	2
2	4
3	6
4	8

Entrada de dados

- Chamamos de entrada de dados o momento em que seu programa recebe dados ou valores por um dispositivo de entrada de dados (como o teclado do computador) ou de um arquivo em disco.

Entrada de dados

A função `input` é utilizada para solicitar dados do usuário.

Exemplo:

```
variavel = input("Texto a ser exibido: ")
```

Entrada de dados

Abra um novo arquivo no IDLE (File → New file ou aperte ctrl+n), digite o exemplo abaixo e rode (Run → Run module ou aperte F5):

```
x = input("Digite um número: ")  
print(x)
```


Entrada de dados

Veja outro exemplo:

```
nome = input("Digite seu nome:")
```

```
print("Você digitou %s" % nome)
```

```
print("Olá, %s!" % nome)
```

Conversão da entrada de dados

A função `input` sempre retorna valores do tipo `string`, ou seja, não importa se digitamos apenas números, o resultado sempre é `string`.

Utilizamos a função `int` para converter o valor retornado em um número inteiro, e a função `float` para convertê-lo em número decimal ou de ponto flutuante.

Conversão da entrada de dados

Exemplo:

```
anos = int(input("Anos de serviço: "))  
valor_por_ano = float(input("Valor por ano: "))  
bônus = anos * valor_por_ano  
print("Bônus de R$ %5.2f" % bônus)
```

Exercício 3.7

- Faça um programa que peça dois números inteiros. Imprima a soma desses dois números na tela.

Exercício 3.8

- Escreva um programa que leia um valor em metros e o exiba convertido em milímetros.

Exercício 3.9

- Escreva um programa que leia a quantidade de dias, horas, minutos e segundos do usuário. Calcule o total em segundos.

Exercício 3.10

- Faça um programa que calcule o aumento de um salário. Ele deve solicitar o valor do salário e a porcentagem do aumento. Exiba o valor do aumento e do novo salário.

Exercício 3.11

- Faça um programa que solicite o preço de uma mercadoria e o percentual de desconto. Exiba o valor do desconto e o preço a pagar.

Exercício 3.12

- Escreva um programa que calcule o tempo de uma viagem de carro.
- Pergunte a distância a percorrer e a velocidade média esperada para a viagem.

Exercício 3.13

- Escreva um programa que converta uma temperatura digitada em °C em °F. A fórmula para essa conversão é:

$$F = \frac{9 \times C}{5} + 32$$

Exercício 3.14

- Escreva um programa que pergunte a quantidade de km percorridos por um carro alugado pelo usuário, assim como a quantidade de dias pelos quais o carro foi alugado. Calcule o preço a pagar, sabendo que o carro custa R\$ 60 por dia e R\$ 0,15 por km rodado

Exercício 3.15

- Escreva um programa para calcular a redução do tempo de vida de um fumante. Pergunte a quantidade de cigarros fumados por dia e quantos anos ele já fumou. Considere que um fumante perde 10 minutos de vida a cada cigarro, calcule quantos dias de vida um fumante perderá. Exiba o total em dias.

Erros

- A entrada de dados é um ponto frágil em nossos programas.
- Como não temos como prever o que o usuário vai digitar, temos que nos preparar.

Erros

```
nome = input("Digite seu nome: ")  
idade = int(input("Digite sua idade: "))  
saldo = float(input("Digite o saldo da sua conta  
bancária: "))  
print(nome)  
print(idade)  
print(saldo)
```

Erros

Digite seu nome: Maria

Digite sua idade: 28

Digite o saldo da sua conta bancária: 34

Maria

28

34.0

Observe que, como utilizamos a função float para converter o saldo, mesmo inserindo 34 o valor foi convertido para 34.0.

Erros

Digite seu nome: Minduim

Digite sua idade: abc

Traceback (most recent call last):

File "input/input2.py", line 2, in <module>

idade = int(input("Digite sua idade: "))

ValueError: invalid literal for int() with base 10: 'abc'

Observe que o erro interrompe nosso programa, exibindo a linha em que ocorreu e o nome do erro. Nesse caso, o erro aconteceu na linha 2 e seu nome é ValueError:

Erros

Digite seu nome: Juanito

Digite sua idade: 31

Digite o saldo da sua conta bancária: abc

Traceback (most recent call last):

File "input/input2.py", line 3, in <module>

```
saldo = float(input("Digite o saldo da sua conta bancária: "))
```

ValueError: could not convert string to float: abc

A entrada de dados é um pouco rústica, parando em caso de erro. Mais adiante, aprenderemos sobre exceções em Python e como tratar esse tipo de erro.

Condições

Nem sempre todas as linhas dos programas serão executadas. Muitas vezes, será mais interessante decidir que partes do programa devem ser executadas com base no resultado de uma condição.

A base dessas decisões consistirá em expressões lógicas que permitam representar escolhas em programas.

if

As condições servem para selecionar quando uma parte do programa deve ser ativada e quando deve ser simplesmente ignorada.

if

if <condição>:
 bloco verdadeiro

O if nada mais é que o nosso “se”. Poderemos então entendê-lo em português da seguinte forma: se a condição for verdadeira, faça alguma coisa.

if

Vejam os um exemplo: ler dois valores e imprimir o maior deles:

```
a = int(input("Primeiro valor: "))
```

```
b = int(input("Segundo valor: "))
```

```
if a > b:
```

```
    print ("O primeiro número é o maior!")
```

```
if b > a:
```

```
    print ("O segundo número é o maior!")
```

Temos a condição $a > b$. Essa expressão será avaliada, e, se o seu resultado for verdadeiro, a linha seguinte será executada. Se for falso, a linha seguinte será ignorada. O mesmo acontece para a condição $b > a$.

if

- As linhas com o “if” foram terminadas com o símbolo dois pontos (:). Quando isso acontece, temos o anúncio de um bloco de linhas a seguir.
- Em Python, um bloco é representado deslocando-se o início da linha para a direita. O bloco continua até a primeira linha com deslocamento diferente.

if

- Python é uma das poucas linguagens de programação que utiliza o deslocamento do texto à direita (recuo) para marcar o início e o fim de um bloco. Outras linguagens contam com palavras especiais para isso, como BEGIN e END, em Pascal; ou as famosas chaves ({ e }), em C e Java.

Exercício

- Analise o programa anterior. Responda o que acontece se o primeiro e o segundo valores forem iguais? Explique.

if

```
idade = int(input("Digite a idade do seu carro: "))  
  
if idade <= 3:  
    print("Seu carro é novo")  
  
if idade > 3:  
    print("Seu carro é velho")
```

Execute várias vezes com as seguintes idades: 1, 3 e 5.

Exercício 4.2

Escreva um programa que pergunte a velocidade do carro de um usuário. Caso ultrapasse 80 km/h, exiba uma mensagem dizendo que o usuário foi multado. Nesse caso, exiba o valor da multa, cobrando R\$ 5 por km/h acima de 80 km/h.

if

- Um bloco de linhas em Python pode ter mais de uma linha, embora o último exemplo mostre apenas dois blocos com uma linha em cada. Se você precisar de duas ou mais no mesmo bloco, escreva essas linhas na mesma direção ou na mesma coluna da primeira linha do bloco. Isso basta para representá-lo.

Exercício 4.3

- Escreva um programa que leia três números e que imprima o maior e o menor.

Exercício 4.4

- Escreva um programa que pergunte o salário do funcionário e calcule o valor do aumento. Para salários superiores a R\$ 2.250,00, calcule um aumento de 10%. Para os inferiores ou iguais, de 15%.

else

- Quando há problemas onde a segunda condição é simplesmente o inverso da primeira, podemos usar outra forma de if para simplificar os programas.
- Essa forma é a cláusula “else”

else

```
idade = int(input("Digite a idade de seu carro:  
"))
```

```
if idade <= 3:
```

```
    print("Seu carro é novo")
```

```
else:
```

```
    print("Seu carro é velho")
```

Veja que utilizamos ":" após else. Isso é necessário porque else inicia um bloco, da mesma forma que if.

else

- É importante notar que devemos escrever else na mesma coluna do if, ou seja, com o mesmo recuo(“indentação” do inglês “indent”). Assim, o interpretador reconhece que else se refere a um determinado if.

else

- A vantagem de usar else é deixar os programas mais claros, uma vez que podemos expressar o que fazer caso a condição especificada em if seja falsa.

Exercício 4.6

- Escreva um programa que pergunte a distância que um passageiro deseja percorrer em km. Calcule o preço da passagem, cobrando R\$ 0,50 por km para viagens de até de 200 km, e R\$ 0,45 para viagens mais longas.

Estruturas aninhadas

- Nem sempre nossos programas serão tão simples. Muitas vezes, precisaremos aninhar vários if para obter o comportamento desejado do programa.
- Aninhar, nesse caso, é utilizar um if dentro de outro.

Estruturas aninhadas

- Vejamos o exemplo de calcular a conta de um telefone celular da empresa Tchau.
- Os planos da empresa Tchau são bem interessantes e oferecem preços diferenciados de acordo com a quantidade de minutos usados por mês.

Tchau

- Abaixo de 200 minutos, a empresa cobra R\$ 0,20 por minuto.
- Entre 200 e 400 minutos, o preço é de R\$ 0,18.
- Acima de 400 minutos, o preço por minuto é de R\$ 0,15.

Tchau

```
minutos=int(input("Quantos minutos você utilizou este mês:"))
if minutos < 200:
    preço = 0.20
else:
    if minutos < 400:
        preço = 0.18
    else:
        preço = 0.15
print("Você vai pagar este mês: R$%6.2f" % (minutos * preço))
```

Tchau

Lembre-se de que o alinhamento do texto é muito importante em Python!

Exemplo 4.7

Faça um programa que leia a categoria de um produto e determine o preço pela tabela:

Categoria	Preço
1	10,00
2	18,00
3	23,00
4	26,00
5	31,00


```
categoria = int(input("Digite a categoria do produto:"))
if categoria == 1:
    preço = 10
else:
    if categoria == 2:
        preço = 18
    else:
        if categoria == 3:
            preço = 23
        else:
            if categoria == 4:
                preço = 26
            else:
                if categoria == 5:
                    preço = 31
                else:
                    print("Categoria inválida, digite um valor entre 1 e 5!")
                    preço = 0
print("O preço do produto é: R$%.2f" % preço)
```

Estruturas aninhadas

- Observe que o alinhamento se tornou um grande problema, uma vez que tivemos que deslocar à direita a cada else.

Estruturas aninhadas

- Neste exemplo também introduzimos o conceito de validação da entrada. Dessa vez, se o usuário digitar um valor inválido, receberá uma mensagem de erro na tela.

Estruturas aninhadas

- Quando lermos um programa com estruturas aninhadas, devemos prestar muita atenção para visualizar corretamente os blocos. Observe como o alinhamento é importante.

Exercício 4.7

- Rastreie o programa anterior e monte uma tabela com as linhas que seriam executadas com os inputs de 1 até 6.

Tabela

Categoria	Linhas executadas
1	1,2,3,19
2	1,2,4,5,6,19
3	1,2,4,5,7,8,9,19
4	1,2,4,5,7,8,10,11,12,19
5	1,2,4,5,7,8,10,11,13,14,15,19
outras	1,2,4,5,7,8,10,11,13,14,16,17,18,19

elif

- Python apresenta uma solução muito interessante ao problema de múltiplos ifs aninhados.
- A cláusula elif substitui um par else if, mas sem criar outro nível de estrutura, evitando problemas de deslocamentos desnecessários à direita.

elif

- Vamos revisar o problema anterior, mas dessa vez usando elif

elif

```
categoria = int(input("Digite a categoria do produto:"))
if categoria == 1:
    preço = 10
elif categoria == 2:
    preço = 18
elif categoria == 3:
    preço = 23
elif categoria == 4:
    preço = 26
elif categoria == 5:
    preço = 31
else:
    print("Categoria inválida, digite um valor entre 1 e 5!")
    preço = 0
print("O preço do produto é: R$%6.2f" % preço)
```

Exercício 4.8

- Escreva um programa que leia dois números e que pergunte qual operação você deseja realizar. Você deve poder calcular a soma (+), subtração (-), multiplicação (*) e divisão (/). Exiba o resultado da operação solicitada.

Exercício 4.9

- Escreva um programa para aprovar o empréstimo bancário para compra de uma casa. O programa deve perguntar o valor da casa a comprar, o salário e a quantidade de anos a pagar. O valor da prestação mensal não pode ser superior a 30% do salário. Calcule o valor da prestação como sendo o valor da casa a comprar dividido pelo número de meses a pagar.

Exercício 4.10

- Escreva um programa que calcule o preço a pagar pelo fornecimento de energia elétrica. Pergunte a quantidade de kWh consumida e o tipo de instalação: R para residências, I para indústrias e C para comércios. Calcule o preço a pagar de acordo com a tabela a seguir.

Exercício 4.10

Preço por tipo e faixa de consumo		
Tipo	Faixa (kWh)	Preço
Residencial	Até 500	R\$ 0,40
	Acima de 500	R\$ 0,65
Comercial	Até 1000	R\$ 0,55
	Acima de 1000	R\$ 0,60
Industrial	Até 5000	R\$ 0,55
	Acima de 5000	R\$ 0,60

Repetições/Laços/Loops

- Repetições são utilizadas para executar a mesma parte de um programa várias vezes, normalmente dependendo de uma condição.

Repetições

Por exemplo, para imprimir três números na tela, poderíamos escrever um programa:

```
print(1)
```

```
print(2)
```

```
print(3)
```

Repetições

Podemos imaginar que para imprimir três números, começando de 1 até o 3, devemos variar `print(x)`, onde `x` varia de 1 a 3:

```
x=1
```

```
print(x)
```

```
x=2
```

```
print(x)
```

```
x=3
```

```
print(x)
```


Repetições

Outra solução seria incrementar o valor de x após cada print.

```
x=1
```

```
print(x)
```

```
x=x+1
```

```
print(x)
```

```
x=x+1
```

```
print(x)
```

Repetições

- Porém, se o objetivo fosse escrever 100 números, a solução não seria tão agradável, pois teríamos que escrever pelo menos 200 linhas!
- A estrutura de repetição aparece para nos auxiliar a resolver esse tipo de problema.

Repetições

- Uma das estruturas de repetição do Python é o `while`, que repete um bloco enquanto a condição for verdadeira.
- Seu formato é onde condição é uma expressão lógica, e bloco representa as linhas de programa a repetir enquanto o resultado da condição for verdadeiro.

Repetições

```
while <condição>:  
    <bloco>
```

Repetições

Para resolver o problema de escrever três números utilizando o while:

```
x=1
```

```
while x<=3:
```

```
    print(x)
```

```
    x = x + 1
```

Exercício 5.1

- Modifique o programa para exibir os números de 1 a 100.

```
x=1
```

```
while x<=3:
```

```
    print(x)
```

```
    x = x + 1
```

Exercício 5.2

- Modifique o programa para exibir os números de 50 a 100.

Exercício 5.3

- Faça um programa para escrever a contagem regressiva do lançamento de um foguete. O programa deve imprimir 10, 9, 8, ..., 1, 0 e Fogo! na tela.

Contadores

- Imagine um problema onde deveríamos imprimir os números inteiros entre 1 e um valor digitado pelo usuário.

Exercício

- Modifique primeiro programa que fizemos com *loops*, de forma que o último número a imprimir seja informado pelo usuário. Ou seja, de 1 até N, aonde N é o número digitado pelo usuário.

Contadores

- Um contador é uma variável utilizada para contar o número de ocorrências de um determinado evento; nesse caso, o número de repetições do while, que satisfaz às necessidades de nosso problema.

Contadores

- Experimente esse programa com vários valores, primeiro digitando 5, depois 500 e, por fim, 0 (zero). Provavelmente, 5 e 500 produzirão os resultados esperados, ou seja, a impressão de 1 até 5, ou de 1 até 500. Porém, quando digitamos zero, nada acontece, e o programa termina logo a seguir, sem impressão.

Exemplo 5.7

- Faça o programa imprimir apenas os números pares entre 0 e um número digitado pelo usuário

Poderíamos resolver o problema com um if para testar se x é par ou ímpar antes de imprimir. Vale lembrar que um número é par quando é 0 ou múltiplo de 2. Quando é múltiplo de 2, temos que o resto da divisão desse número por 2 é 0, ou seja, o resultado é uma divisão exata, sem resto. Em Python, podemos escrever esse teste com `x % 2 == 0`

Exemplo 5.7

```
fim=int(input("Digite o último número a imprimir:"))
```

```
x = 0
```

```
while x <= fim:
```

```
    if x % 2 == 0:
```

```
        print(x)
```

```
    x = x + 1
```

Exemplo 5.8

- Poderíamos resolvê-lo de forma ainda mais simples se adicionássemos 2 a x a cada repetição. Isso garantiria que x sempre fosse par.

```
fim=int(input("Digite o último número a imprimir:"))
```

```
x = 0
```

```
while x <= fim:
```

```
    print(x)
```

```
    x = x + 2
```


Exercício 5.4

- Modifique o programa anterior para imprimir de 1 até o número digitado pelo usuário, mas, dessa vez, apenas os números ímpares.

Exercício 5.5

- Reescreva o programa anterior para escrever os 10 primeiros múltiplos de 3.

Contadores

- Vejamos outro tipo de problema. Imagine ter que imprimir a tabuada de adição de um número digitado pelo usuário. Essa tabuada deve ser impressa de 1 a 10, sendo n o número digitado pelo usuário. Teríamos, assim, $n+1$, $n+2$, ... $n+10$.

Contadores

```
n = int(input("Tabuada de:"))
```

```
x = 1
```

```
while x <= 10:
```

```
    print(n+x)
```

```
    x=x+1
```

Exercício 5.6

- Altere o programa anterior para exibir os resultados no mesmo formato de uma tabuada:
 $2 \times 1 = 2$, $2 \times 2 = 4$, ...

Exercício 5.7

- Modifique o programa anterior de forma que o usuário também digite o início e o fim da tabuada, em vez de começar com 1 e 10.

Exercício 5.8

- Escreva um programa que leia dois números. Imprima o resultado da multiplicação do primeiro pelo segundo. Utilize apenas os operadores de soma e subtração para calcular o resultado. Lembre-se de que podemos entender a multiplicação de dois números como somas sucessivas de um deles. Assim, $4 \times 5 = 5 + 5 + 5 + 5 = 4 + 4 + 4 + 4 + 4$.

Exercício 5.9

- Escreva um programa que leia dois números. Imprima a divisão inteira do primeiro pelo segundo, assim como o resto da divisão. Utilize apenas os operadores de soma e subtração para calcular o resultado. Lembre-se de que podemos entender o quociente da divisão de dois números como a quantidade de vezes que podemos retirar o divisor do dividendo. Logo, $22 \div 4 = 5$, uma vez que podemos subtrair 4 cinco vezes de 22, com resto 2.

Contadores

- Contadores também podem ser úteis quando usados com condições dentro dos programas. Vejamos um programa para corrigir um teste de múltipla escolha com três questões.

Contadores

A resposta da primeira é “b”; da segunda, “a”; e da terceira, “d”:

```
pontos = 0
```

```
questão = 1
```

```
while questão <= 3:
```

```
    resposta = input("Resposta da questão %d: " % questão)
```

```
    if questão == 1 and resposta == "b":
```

```
        pontos = pontos + 1
```

```
    if questão == 2 and resposta == "a":
```

```
        pontos = pontos + 1
```

```
    if questão == 3 and resposta == "d":
```

```
        pontos = pontos + 1
```

```
    questão +=1
```

```
print("O aluno fez %d ponto(s)" % pontos)
```

Execute o programa e digite todas as respostas corretas, depois tente com respostas diferentes. Veja que estamos verificando apenas respostas simples de uma só letra

Contadores

- Em Python, uma letra minúscula é diferente de uma maiúscula. Se você digitar “A” na segunda questão em vez de “a”, o programa não considerará essa resposta correta. Uma solução para esse tipo de problema é utilizar o operador lógico or e verificar a resposta maiúscula e minúscula. Por exemplo, questão == 1 and (resposta == "b" or resposta == "B").

Exercício 5.10

- Modifique o programa anterior para que aceite respostas com letras maiúsculas e minúsculas em todas as questões.

Acumuladores

- Nem só de contadores precisamos. Em programas para calcular o total de uma soma, por exemplo, precisaremos de acumuladores. A diferença entre um contador e um acumulador é que nos contadores o valor adicionado é constante e, nos acumuladores, variável.

Acumuladores

Vejam os um programa que calcule a soma de 10 números

```
n = 1
```

```
soma = 0
```

```
while n <= 10:
```

```
    x = int(input("Digite o %d número:"%n))
```

```
    soma = soma + x # acumulador
```

```
    n = n + 1 # contador
```

```
print("Soma: %d"%soma)
```

Acumuladores

- Podemos definir a média aritmética como a soma de vários números divididos pela quantidade de números somados. Assim, se somarmos três números, 4, 5 e 6, teríamos a média aritmética como $(4+5+6) / 3$, onde 3 é a quantidade de números. Se chamarmos o primeiro número de $n1$, o segundo de $n2$, e o terceiro de $n3$, teremos $(n1 + n2 + n3) / 3$.

Acumuladores

- Vejamos um programa que calcula a média de cinco números digitados pelo usuário

$$média = (n1+n2+n3+n4+n5)/5 = \frac{n1 + n2 + n3 + n4 + n5}{5}$$

Acumuladores

Em vez de utilizarmos cinco variáveis, vamos acumular os valores à medida que são lidos.

```
x = 0
```

```
soma = 0
```

```
while x < 5:
```

```
    n = int(input("Digite o número %d:" % (x+1)))
```

```
    soma = soma + n # acumulador
```

```
    x = x + 1 # contador
```

```
print("Média: %5.2f" % (soma/x))
```

Exercício 5.11

- Escreva um programa que pergunte o depósito inicial e a taxa de juros de uma poupança. Exiba os valores mês a mês para os 24 primeiros meses.
- Escreva o total ganho com juros no período

Exercício 5.12

- Altere o programa anterior de forma a perguntar também o valor depositado mensalmente. Esse valor será depositado no início de cada mês, e você deve considerá-lo para o cálculo de juros do mês seguinte.

Exercício 5.13

- Escreva um programa que pergunte o valor inicial de uma dívida e o juro mensal. Pergunte também o valor mensal que será pago. Imprima o número de meses para que a dívida seja paga, o total pago e o total de juros pago.
 - Verifique se o pagamento é inferior ou igual aos juros * dívida, se for, significa que a dívida nunca será paga
 - Para cada mês, calcule o valor a ser pago em juros, some isso a dívida e depois desconte o pagamento
 - Incremente o mês e exiba no final
 - Acumule o total de juros pago e exiba no final

Interrompendo a repetição

- Embora muito útil, a estrutura while só verifica sua condição de parada no início de cada repetição. Dependendo do problema, a habilidade de terminar while dentro do bloco a repetir pode ser interessante.

Interrompendo a repetição

- A instrução `break` é utilizada para interromper a execução de `while` independentemente do valor atual de sua condição.

Interrompendo a repetição

Vejamos o exemplo da leitura de valores até que digitemos 0 (zero)

```
s=0
```

```
while True:
```

```
    v=int(input("Digite um número a somar ou 0 para sair:"))
```

```
    if v==0:
```

```
        break
```

```
    s = s+v
```

```
print(s)
```

Exercício 5.14

- Escreva um programa que leia números inteiros do teclado. O programa deve ler os números até que o usuário digite 0 (zero). No final da execução, exiba a quantidade de números digitados, assim como a soma e a média aritmética.

Exercício 5.15

Escreva um programa para controlar uma pequena máquina registradora. Você deve solicitar ao usuário que digite o código do produto e a quantidade comprada. Utilize a tabela de códigos abaixo para obter o preço de cada produto:

Código	Preço
1	0,50
2	1,00
3	4,00
5	7,00
9	8,00

Seu programa deve exibir o total das compras depois que o usuário digitar 0. Qualquer outro código deve gerar a mensagem de erro “Código inválido”.

Interrompendo a repetição

- Vejamos como exemplo um programa que leia um valor e que imprima a quantidade de cédulas necessárias para pagar esse mesmo valor. Para simplificar, vamos trabalhar apenas com valores inteiros e com cédulas de R\$ 50, R\$ 20, R\$ 10, R\$ 5 e R\$ 1.

```
valor=int(input("Digite o valor a pagar:"))
cédulas=0
atual=50
apagar=valor
while True:
    if atual<=apagar:
        apagar-=atual
        cédulas+=1
    else:
        print("%d cédula(s) de R$%d" % (cédulas, atual))
        if apagar == 0:
            break
        if atual == 50:
            atual = 20
        elif atual == 20:
            atual = 10
        elif atual == 10:
            atual = 5
        elif atual == 5:
            atual = 1
        cédulas = 0
```

Exercício 5.16

- Execute o programa anterior para os seguintes valores: 501, 745, 384, 2, 7 e 1.

Exercício 5.17

- O que acontece se digitarmos 0 (zero) no valor a pagar?

Exercício 5.17

Digite o valor a pagar:0

0 cédula(s) de R\$50

Exercício 5.18

- Modifique o programa para também trabalhar com notas de R\$ 100.

Exercício 5.19

- Modifique o programa para aceitar valores decimais, ou seja, também contar moedas de 0,01, 0,02, 0,05, 0,10 e 0,50.

Exercício 5.20

- O que acontece se digitarmos 0,001 no programa anterior? Caso ele não funcione, altere-o de forma a corrigir o problema.

Exercício 5.20

0 moeda(s) de R\$ 0.01

0 moeda(s) de R\$ 0.01

0 moeda(s) de R\$ 0.01

0 moeda(s) de R\$ 0.01

0 moeda(s) de R\$ 0.01

0 moeda(s) de R\$ 0.01

0 moeda(s) de R\$ 0.01

0 moeda(s) de R\$ 0.01

...

Isso é devido a uma limitação do ponto flutuante, pois nem todos os valores podem ser expressos em binário.

ex:

```
>>> 1.1 - 1 - 0.1
```

```
8.326672684688674e-17
```

Exercício 5.20

Para solucionar isso, podemos usar round()

```
apagar = round(apagar - atual, 2)
```

```
...
```

```
if round(apagar, 2) == 0:
```

Ou comparar com um valor menor que 0.01

```
if apagar < 0.01:
```

Repetições aninhadas

- Podemos combinar vários while de forma a obter resultados mais interessantes, como a repetição com incremento de duas variáveis.

Repetições aninhadas

- Imagine imprimir as tabuadas de multiplicação de 1 a 10. Vejamos como fazer isso

Repetições aninhadas

```
tabuada=1
while tabuada <= 10:
    número = 1
    while número <= 10:
        print("%d x %d = %d" % (tabuada, número, tabuada *
número))
        número+=1
    tabuada+=1
```

Repetições aninhadas

- Vejamos o mesmo problema, mas sem utilizar repetições aninhadas

Repetições aninhadas

```
tabuada=1
```

```
número=1
```

```
while tabuada <= 10:
```

```
    print("%d x %d = %d" % (tabuada, número, tabuada * número))
```

```
    número+=1
```

```
    if número == 11:
```

```
        número = 1
```

```
        tabuada+=1
```


Exercício 5.21

- Reescreva o programa ‘das cédulas’ de forma a continuar executando até que o valor digitado seja 0. Utilize repetições aninhadas.

Exercício 5.22

- Escreva um programa que exiba uma lista de opções (menu): adição, subtração, divisão, multiplicação e sair. Imprima a tabuada da operação escolhida. Repita até que a opção saída seja escolhida.

Exercício 5.23

- Escreva um programa que leia um número e verifique se é ou não um número primo. Para fazer essa verificação, calcule o resto da divisão do número por 2 e depois por todos os números ímpares até o número lido. Se o resto de uma dessas divisões for igual a zero, o número não é primo. Observe que 0 e 1 não são primos e que 2 é o único número primo que é par.

Exercício 5.24

- Modifique o programa anterior de forma a ler um número n . Imprima os n primeiros números primos.

Exercício 5.25

- Escreva um programa que calcule a raiz quadrada de um número.
- Utilize o método de Newton para obter um resultado aproximado. Sendo n o número a obter a raiz quadrada, considere a base $b=2$. Calcule p usando a fórmula $p=(b+(n/b))/2$. A cada passo, faça $b=p$ e recalcule p usando a fórmula apresentada. Pare quando a diferença absoluta entre n e o quadrado de p for menor que 0,0001.

Exercício 5.26

- Escreva um programa que calcule o resto da divisão inteira entre dois números. Utilize apenas as operações de soma e subtração para calcular o resultado.

Exercício 5.27

- Escreva um programa que verifique se um número é palíndromo.
- Um número é palíndromo se continua o mesmo caso seus dígitos sejam invertidos.
- Exemplos: 454, 10501

Listas

- Listas são um tipo de variável que permite o armazenamento de vários valores, acessados por um índice.

Lista vazia

$L = []$

Uma lista com três elementos

$Z = [15, 8, 9]$

Acesso a uma lista

```
>>> Z=[15,8,9]
```

```
>>> Z[0]
```

```
15
```

```
>>> Z[1]
```

```
8
```

```
>>> Z[2]
```

```
9
```

Listas

- Uma lista pode conter zero ou mais elementos de um mesmo tipo ou de tipos diversos, podendo inclusive conter outras listas.

Listas

```
>>> a = [ 1, 'str', 2.3 ]
```

```
>>> a[0]
```

```
1
```

```
>>> type(a[0])
```

```
<class 'int'>
```

```
>>> type(a[1])
```

```
<class 'str'>
```

```
>>> type(a[2])
```

```
<class 'float'>
```

Modificação de uma lista

```
>>> Z=[15,8,9]
```

```
>>> Z[0]
```

```
15
```

```
>>> Z[0]=7
```

```
>>> Z[0]
```

```
7
```

```
>>> Z
```

```
[7, 8, 9]
```

Cálculo da média

```
notas=[6,7,5,8,9]
```

```
soma=0
```

```
x=0
```

```
while x<5:
```

```
    soma += notas[x]
```

```
    x+=1
```

```
print("Média: %5.2f" % (soma/x))
```

Cálculo da média com notas digitadas

```
notas=[0,0,0,0,0]
x=0
while x<5:
    notas[x]=float(input("Nota %d:" % x))
    x+=1
x=0
soma=0
while x<5:
    print("Nota %d: %6.2f" % (x, notas[x]))
    soma += notas[x]
    x+=1
print("Média: %5.2f" % (soma/x))
```


Exercício 6.1

- Modifique o programa anterior para ler 7 notas em vez de 5.

Trabalhando com índices

- Vejamos outro exemplo: um programa que lê cinco números, armazena-os em uma lista e depois solicita que o usuário escolha um número a mostrar. O objetivo é, por exemplo, ler 15, 12, 5, 7 e 9 e armazená-los na lista. Depois, se o usuário digitar 2, ele imprimirá o segundo número digitado, 3, o terceiro, e assim sucessivamente. Observe que o índice do primeiro número é 0, e não 1: essa pequena conversão será feita no programa

Apresentação de números

```
números=[0,0,0,0,0]
```

```
x=0
```

```
while x<5:
```

```
    números[x]=int(input("Número %d:" % (x+1)))
```

```
    x+=1
```

```
while True:
```

```
    escolhido=int(input("Que posição você quer imprimir (0 para sair): "))
```

```
    if escolhido == 0:
```

```
        break
```

```
    print("Você escolheu o número: %d" % (números[escolhido-1]))
```

Copia de listas

```
>>> L=[1,2,3,4,5]
```

```
>>> V=L
```

```
>>> L
```

```
[1, 2, 3, 4, 5]
```

```
>>> V
```

```
[1, 2, 3, 4, 5]
```

```
>>> V[0]=6
```

```
>>> V
```

```
[6, 2, 3, 4, 5]
```

```
>>> L
```

```
[6, 2, 3, 4, 5]
```

Copia de listas

- Veja que, ao modificarmos `V`, modificamos também o conteúdo de `L`. Isso porque uma lista em Python é um objeto e, quando atribuímos um objeto a outro, estamos apenas copiando a mesma referência da lista, e não seus dados em si. Nesse caso, `V` funciona como um apelido de `L`, ou seja, `V` e `L` são a mesma lista.

Copia de listas

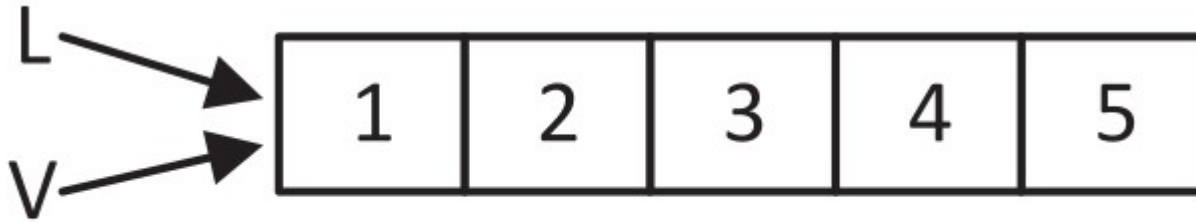


Figura 6.1 – Duas variáveis referenciando a mesma lista.

Quando modificamos $V[0]$, estamos modificando o mesmo valor de $L[0]$, pois ambos são referências, ou apelidos para a mesma lista na memória.

Cópia de listas

- Para criar uma cópia independente de uma lista, utilizaremos outra sintaxe.

Cópia de listas

```
>>> L=[1,2,3,4,5]
```

```
>>> V=L[:]
```

```
>>> V[0]=6
```

```
>>> L
```

```
[1, 2, 3, 4, 5]
```

```
>>> V
```

```
[6, 2, 3, 4, 5]
```

Ao escrevermos `L[:]`, estamos nos referindo a uma nova cópia de `L`.

Copia de listas

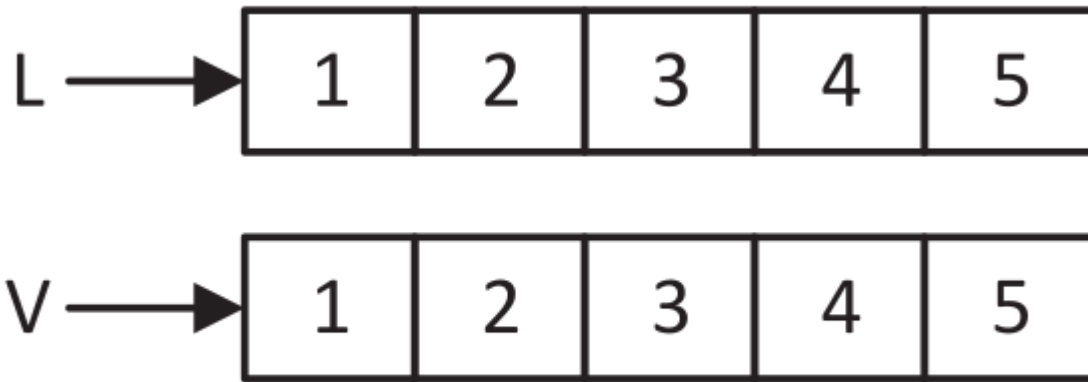


Figura 6.2 – Duas variáveis referenciando duas listas.

Copia de listas

```
V = L.copy()
```

```
V = list(L)
```

```
import copy
```

```
new_list = copy.copy(old_list)
```

Fatiamento de listas

- Podemos também fatiar uma lista, da mesma forma que fizemos com strings

Fatiamento de listas

```
>>> L=[1,2,3,4,5]
```

```
>>> L[0:5]
```

```
[1, 2, 3, 4, 5]
```

```
>>> L[:5]
```

```
[1, 2, 3, 4, 5]
```

```
>>> L[:-1]
```

```
[1, 2, 3, 4]
```

```
>>> L[1:3]
```

```
[2, 3]
```

Fatiamento de listas

```
>>> L[1:4]
```

```
[2, 3, 4]
```

```
>>> L[3:]
```

```
[4, 5]
```

```
>>> L[:3]
```

```
[1, 2, 3]
```

```
>>> L[-1]
```

```
5
```

```
>>> L[-2]
```

```
4
```

Veja que índices negativos também funcionam. Um índice negativo começa a contar do último elemento, mas observe que começamos de -1. Assim L[0] representa o primeiro elemento; L[-1], o último; L[-2], o penúltimo, e assim por diante.

Tamanho de listas

- Podemos usar a função `len` com listas. O valor retornado é igual ao número de elementos da lista.

Tamanho de listas

```
>>> L=[12,9,5]
```

```
>>> len(L)
```

```
3
```

```
>>> V=[]
```

```
>>> len(V)
```

```
0
```

Tamanho de listas

- A função len pode ser utilizada em repetições para controlar o limite dos índices.

Tamanho de listas

```
L=[1,2,3]
```

```
x=0
```

```
while x < len(L):
```

```
    print(L[x])
```

```
    x+=1
```

Tamanho de listas

- A vantagem é que se trocarmos L para:
- $L=[7,8,9,10,11,12]$
- O resto do programa continuaria funcionando, pois utilizamos a função len para calcular o tamanho da lista.

Tamanho de listas

- Observe que o valor retornado pela função `len` é um número que não pode ser utilizado como índice, mas que é perfeito para testarmos os limites de uma lista
- Isso acontece porque `len` retorna a quantidade de elementos na lista e nossos índices começam a ser numerados de 0 (zero). Assim, os índices válidos de uma lista (`L`) variam de 0 até o valor de `len(L)-1`.

Adição de elementos

- Uma das principais vantagens de trabalharmos com listas é poder adicionar novos elementos durante a execução do programa.

Adição de elementos

```
>>> L=[]  
>>> L.append("a")  
>>> L  
['a']  
>>> L.append("b")  
>>> L  
['a', 'b']  
>>> L.append("c")  
>>> L  
['a', 'b', 'c']  
>>> len(L)  
3
```

Adição de elementos

- Para adicionar um elemento ao fim da lista, utilizaremos o método `append`. Em Python, chamamos um método escrevendo o nome dele após o nome do objeto.

Adição de elementos

- Como listas são objetos, sendo L a lista, teremos `L.append(valor)`.
- Métodos são recursos de orientação a objetos, suportados e muito usados em Python.

Adição de elementos

- Você pode imaginar um método como uma função do objeto.

Adição de elementos

- Vejamos um programa que lê números até que 0 seja digitado. Esse programa depois os imprimirá na mesma ordem em que foram digitados

Adição de elementos

```
L=[]  
while True:  
    n=int(input("Digite um número (0 sai):"))  
    if n == 0:  
        break  
    L.append(n)  
x=0  
while x < len(L):  
    print(L[x])  
    x=x+1
```

Adição de elementos

- Outra forma de adicionarmos elementos a uma lista é com adição de listas

Adição de elementos

```
>>> L=[]
```

```
>>> L=L+[1]
```

```
>>> L
```

```
[1]
```

```
>>> L+= [2]
```

```
>>> L
```

```
[1, 2]
```

```
>>> L+= [3,4,5]
```

```
>>> L
```

```
[1, 2, 3, 4, 5]
```

Adição de elementos

- Quando adicionamos apenas um elemento, tanto `L.append(1)` quanto `L+[1]` produzem o mesmo resultado.
- Isso porque, quando adicionamos uma lista a outra, o interpretador executa um método chamado `extend` que adiciona os elementos de uma lista a outra.

Adição de elementos

```
>>> L=["a"]
>>> L.append("b")
>>> L
['a', 'b']
>>> L.extend(["c"])
>>> L
['a', 'b', 'c']
>>> L.append(["d","e"]) # ← Append!
>>> L
['a', 'b', 'c', ['d', 'e']]
>>> L.extend(["f","g","h"])
>>> L
['a', 'b', 'c', ['d', 'e'], 'f', 'g', 'h']
```

Adição de elementos

- O método `extend` sequer aceita parâmetros que não sejam listas. Se você utilizar o método `append` com uma lista como parâmetro, em vez de adicionar os elementos no fim da lista, `append` adicionará a lista inteira, mas como apenas um novo elemento. Teremos então listas dentro de listas

Adição de elementos

```
>>> L=["a"]
```

```
>>> L.append(["b"])
```

```
>>> L.append(["c","d"])
```

```
>>> len(L)
```

```
3
```

```
>>> L[1]
```

```
['b']
```

```
>>> L[2]
```

```
['c', 'd']
```

```
>>> len(L[2])
```

```
2
```

```
>>> L[2][1]
```

```
'd'
```


Adição de elementos

- Esse conceito é interessante, pois permite a utilização de estruturas de dados mais complexas, como matrizes, árvores e registros. Por enquanto, vamos utilizar esse recurso para armazenar múltiplos valores por elemento.

Exercício 6.2

- Faça um programa que leia duas listas e que gere uma terceira com os elementos das duas primeiras.

Exercício 6.3

- Faça um programa que percorra duas listas e gere uma terceira sem elementos repetidos.

Remoção de elementos da lista

- Como o tamanho da lista pode variar, permitindo a adição de novos elementos, podemos também retirar alguns elementos da lista, ou mesmo todos eles. Para isso, utilizaremos a instrução del

Remoção de elementos da lista

```
>>> L=["a","b","c"]
```

```
>>> del L[1]
```

```
>>> L
```

```
['a', 'c']
```

```
>>> del L[0]
```

```
>>> L
```

```
['c']
```

É importante notar que o elemento excluído não ocupa mais lugar na lista, fazendo

com que os índices sejam reorganizados, ou melhor, que passem a ser calculados

Remoção de elementos da lista

- Podemos também apagar fatias inteiras de uma só vez

Remoção de elementos da lista

```
>>> L=list(range(101))
```

```
>>> del L[1:99]
```

```
>>> L
```

```
[0, 99, 100]
```

Usando listas como filas

- Uma lista pode ser utilizada como fila se obedecermos a certas regras de inclusão e eliminação de elementos. Em uma fila, a inclusão é sempre realizada no fim, e as remoções são feitas no início. Dizemos que o primeiro a chegar é o primeiro a sair (FIFO - First In First Out).

Usando listas como filas

- É mais simples de entender se imaginarmos uma fila de banco. Quando a agência abre pela manhã, a fila está vazia. Quando os clientes começam a chegar, eles vão diretamente para o fim da fila.
- Os caixas então começam a atender esses clientes por ordem de chegada, ou seja, o cliente que chegou primeiro será atendido primeiro. Uma vez que o cliente é atendido, ele sai da fila. Então, um novo cliente passa a ser o primeiro da fila e o próximo a ser atendido.

Usando listas como filas

- Para escrevermos algo similar em Python, imaginaremos uma lista de clientes, representando a fila, onde o valor de cada elemento é igual à ordem de chegada do cliente.

Usando listas como filas

- Se outro cliente chegar, realizaremos um `append` para que ele seja inserido no fim da fila (`fila.append(último)`). Para retirarmos um cliente da fila e atendê-lo, poderíamos fazer `del fila[0]`, porém, isso apagaria o cliente da fila. Se quisermos retirá-lo da fila e, ao mesmo tempo, obter o elemento retirado, podemos utilizar o método `pop` `fila.pop(0)`.

Usando listas como filas

```
último = 10
fila = list(range(1,último+1))
while True:
    print("\nExistem %d clientes na fila" % len(fila))
    print("Fila atual:", fila)
    print("Digite F para adicionar um cliente ao fim da fila,")
    print("ou A para realizar o atendimento. S para sair.")
    operação = input("Operação (F, A ou S):")
    if operação == "A":
        if(len(fila))>0:
            atendido = fila.pop(0)
            print("Cliente %d atendido" % atendido)
        else:
            print("Fila vazia! Ninguém para atender.")
    elif operação == "F":
        último+=1 # Incrementa o ticket do novo cliente
        fila.append(último)
    elif operação == "S":
        break
    else:
        print("Operação inválida! Digite apenas F, A ou S!")
```

Exercício 6.4

- O que acontece quando não verificamos se a lista está vazia antes de chamarmos o método pop?

Exercício 6.4

- `>>> a = []`
- `>>> a.pop(0)`
- Traceback (most recent call last):
- File "<stdin>", line 1, in <module>
- IndexError: pop from empty list
- `>>>`

Exercício 6.5

- Altere o programa da listagem 6.21 de forma a poder trabalhar com vários comandos digitados de uma só vez. Atualmente, apenas um comando pode ser inserido por vez. Altere-o de forma a considerar operação como uma string.
- Exemplo: FFFAAAS significaria três chegadas de novos clientes, três atendimentos e, finalmente, a saída do programa.

Uso de listas como pilhas

- Uma pilha tem uma política de acesso bem definida: novos elementos são adicionados ao topo. A retirada de elementos também é feita pelo topo.

Uso de listas como pilhas

- Imagine uma pilha de pratos para lavar. Retiramos o prato que está no topo da pilha para lavar e, se mais alguns pratos chegarem, serão também adicionados ou empilhados ao topo, ou seja, um sobre o outro.
- Na pilha, o último elemento a chegar é o primeiro a sair (LIFO - Last In First Out).

Uso de listas como pilhas

```
prato = 5
pilha = list(range(1,prato+1))
while True:
    print("\nExistem %d pratos na pilha" % len(pilha))
    print("Pilha atual:", pilha)
    print("Digite E para empilhar um novo prato,")
    print("ou D para desempilhar. S para sair.")
    operação = input("Operação (E, D ou S):")
    if operação == "D":
        if(len(pilha))>0:
            lavado = pilha.pop(-1)
            print("Prato %d lavado" % lavado)
        else:
            print("Pilha vazia! Nada para lavar.")
    elif operação == "E":
        prato+=1 # Novo prato
        pilha.append(prato)
    elif operação == "S":
        break
    else:
        print("Operação inválida! Digite apenas E, D ou S!")
```

Exercício 6.7

Faça um programa que leia uma expressão com parênteses. Usando pilhas(stack), verifique se os parênteses foram abertos e fechados na ordem correta.

Exemplo:

`()`OK

`()()()`OK

`)`Erro

Pesquisa

- Podemos pesquisar se um elemento está ou não em uma lista, verificando do primeiro ao último elemento se o valor procurado estiver presente.

Pesquisa

```
L=[15,7,27,39]
p=int(input("Digite o valor a procurar:"))
achou=False ❶
x=0
while x<len(L):
    if L[x]==p:
        achou=True ❷
        break ❸
    x+=1
if achou: ❹
    print("%d achado na posição %d" % (p,x))
else:
    print("%d não encontrado" % p)
```

Exercício 6.8

- Modifique o primeiro exemplo anterior de forma a realizar a mesma tarefa, mas sem utilizar a variável `achou`. Dica: observe a condição de saída do `while`.

Exercício 6.9

- Modifique o exemplo para pesquisar dois valores. Em vez de apenas p , leia outro valor v que também será procurado. Na impressão, indique qual dos dois valores foi achado primeiro.

Exercício 6.10

- Modifique o programa do exercício 6.9 de forma a pesquisar p e v em toda a lista e informando o usuário a posição onde p e a posição onde v foram encontrados.

Usando for

- Python apresenta uma estrutura de repetição especialmente projetada para percorrer listas. A instrução for funciona de forma parecida a while, mas a cada repetição utiliza um elemento diferente da lista.

Usando for

```
L=[8,9,15]
```

```
for e in L:
```

```
    print(e)
```

A cada repetição, o próximo elemento da lista é utilizado, o que se repete até o fim da lista.

Pesquisa usando for

```
L=[7,9,10,12]
p=int(input("Digite um número a pesquisar:"))
for e in L:
    if e == p:
        print("Elemento encontrado!")
        break ❶
else: ❷
    print("Elemento não encontrado.")
```

Exercício 6.11

- Modifique o programa da listagem 6.15 usando for. Explique por que nem todos os while podem ser transformados em for.

listagem 6.15

```
L=[]  
while True:  
    n=int(input("Digite um número (0 sai):"))  
    if n == 0:  
        break  
    L.append(n)  
x=0  
while x < len(L):  
    print(L[x])  
    x=x+1
```

Range

- Podemos utilizar a função range para gerar listas simples. A função range não retorna uma lista propriamente dita, mas um gerador ou generator.

Range

```
for v in range(10):  
    print(v)
```

Execute o programa e veja o resultado. A função range gerou números de 0 a 9 porque passamos 10 como parâmetro.

Range

- Ela normalmente gera valores a partir de 0, logo, ao especificarmos apenas 10, estamos apenas informando onde parar.
- Experimente mudar de 10 para 20. O programa deve imprimir de 0 a 19.

Range

- A vantagem de utilizarmos a função range é gerar listas eficientemente.

Range

- Com a mesma função range, podemos também indicar qual é o primeiro número a gerar. Para isso, utilizaremos dois parâmetros: início e fim

Range

```
for v in range(5, 8):  
    print(v)
```

Usando 5 como início e 8 como fim, vamos imprimir os números 5, 6 e 7. A nota-

ção aqui para o fim é a mesma utilizada com fatias, ou seja, o fim é um intervalo aberto, isto é, não incluso na faixa de valores.

Range

- Se acrescentarmos um terceiro parâmetro à função range, teremos como saltar entre os valores gerados, por exemplo, `range(0,10,2)` gera os pares entre 0 e 10, pois começa de 0 e adiciona 2 a cada elemento

Range

```
for t in range(3,33,3):  
    print(t, end=" ")  
print()
```

Observe que utilizamos uma construção especial com a função print, onde t é o valor que queremos imprimir, mas com end=" ", que indica a função para não pular de linha após a impressão. end é, na realidade, um parâmetro opcional da função print.

Range

- Observe que um gerador como o retornado pela função `range` não é exatamente uma lista. Embora seja usado de forma parecida, é, na realidade, um objeto de outro tipo. Para transformar um gerador em lista, utilize a função `list`

Range

```
L=list(range(100,1100,50))
```

```
print(L)
```

Enumerate

- Com a função enumerate podemos ampliar as funcionalidades de for facilmente.
- Vejamos como imprimir uma lista, onde teremos o índice entre colchetes e o valor à sua direita

Enumerate

```
L=[5,9,13]
```

```
x=0
```

```
for e in L:
```

```
    print("[%d] %d" % (x,e))
```

```
    x+=1
```

Enumerate

- Veja o mesmo programa, mas utilizando a função enumerate na listagem

Enumerate

```
L=[5,9,13]
```

```
for x, e in enumerate(L):
```

```
    print("[%d] %d" % (x,e))
```

Enumerate

- A função enumerate gera uma tupla em que o primeiro valor é o índice e o segundo é o elemento da lista sendo enumerada. Ao utilizarmos x, e em for, indicamos que o primeiro valor da tupla deve ser colocado em x, e o segundo, em e. Assim, na primeira iteração teremos a tupla (0,5), onde $x=0$ e $e=5$.

Enumerate

- Isso é possível porque Python permite o desempacotamento de valores de uma tupla, atribuindo um elemento da tupla a cada variável em for. O que temos a cada iteração de for é equivalente a $x, e = (0, 5)$, em que o gerador enumerate retorna cada vez uma nova tupla. Os próximos valores retornados são $(1, 9)$ e $(2, 13)$, respectivamente.

Enumerate

- Experimente substituir x,e na listagem 6.30 por z. Antes de print, faça $x,e = z$. Adicione mais um print para exibir também o valor de z.

Operações com listas

- Podemos percorrer uma lista de forma a verificar o menor e o maior valor

Operações com listas

```
L=[1,7,2,4]
máximo=L[0] ❶
for e in L:
    if e > máximo:
        máximo = e
print(máximo)
```

Em ❶, utilizamos um pequeno truque, inicializando o máximo com o valor do primeiro elemento. Precisamos de um valor para máximo antes de utilizá-lo na comparação com if. Se usássemos 0, não teríamos problema, desde que nossa lista não tenha valores negativos.

Exercício 6.12

- Altere o programa da listagem 6.33 de forma a imprimir o menor elemento da lista.

Exercício 6.13

- A lista de temperaturas de Mons, na Bélgica, foi armazenada na lista $T = [-10, -8, 0, 1, 2, 5, -2, -4]$. Faça um programa que imprima a menor e a maior temperatura, assim como a temperatura média.

Aplicações

- Vejamos uma situação na qual temos que selecionar os elementos de uma lista de forma a copiá-los para outras duas listas. Para simplificar o problema, imagine que os valores estejam inicialmente na lista V, mas que devam ser copiados para a P, se forem pares; ou para a I, se forem ímpares.

Cópia de elementos para outras listas

```
V=[9,8,7,12,0,13,21]
```

```
P=[]
```

```
I=[]
```

```
for e in V:
```

```
    if e % 2 == 0:
```

```
        P.append(e)
```

```
    else:
```

```
        I.append(e)
```

```
print("Pares: ", P)
```

```
print("Impares: ", I)
```

Controle da utilização de salas de um cinema

- Vejamos agora um programa que controla a utilização das salas de um cinema. Imagine que a lista Salas = [10,2,1,3,0] contenha o número de lugares vagos nas salas 1, 2, 3, 4 e 5, respectivamente. Esse programa lerá o número da sala e a quantidade de lugares solicitados. Ele deve informar se é possível vender o número de lugares solicitados, ou seja, se ainda há lugares livres. Caso seja possível vender os bilhetes, atualizará o número de lugares livres.

```
lugares_vagos=[10,2,1,3,0]
while True:
    sala=int(input("Sala (0 sai): "))
    if sala == 0:
        print("Fim")
        break
    if sala>len(lugares_vagos) or sala<1:
        print("Sala inválida")
    elif lugares_vagos[sala-1]==0:
        print("Desculpe, sala lotada!")
    else:
        lugares =int(input("Quantos lugares você deseja (%d vagos):"
            % lugares_vagos[sala-1]))
        if lugares > lugares_vagos[sala-1]:
            print("Esse número de lugares não está disponível.")
        elif lugares < 0:
            print("Número inválido")
        else:
            lugares_vagos[sala-1]-=lugares
            print("%d lugares vendidos" % lugares)
print("Utilização das salas")
for x,l in enumerate(lugares_vagos):
    print("Sala %d - %d lugar(es) vazio(s)" % (x+1, l))
```

Listas com strings

- No capítulo 3 vimos que strings podem ser indexadas ou acessadas letra por letra.
- Listas em Python funcionam da mesma forma, permitindo o acesso a vários valores e se tornando uma das principais estruturas de programação da linguagem.

Listas com strings

- Vejamos agora outro exemplo de listas, mas utilizando strings

Listas com strings

```
>>> S=["maças", "peras", "kiwis"]
```

```
>>> print(len(S))
```

```
3
```

```
>>> print(S[0])
```

```
maças
```

```
>>> print(S[1])
```

```
peras
```

```
>>> print(S[2])
```

```
kiwis
```

Lendo e imprimindo uma lista de compras

```
compras=[]  
while True:  
    produto=input("Produto:")  
    if produto == "fim":  
        break  
    compras.append(produto)  
for p in compras:  
    print(p)
```

Listas dentro de listas

- Um fator interessante é que podemos acessar as strings dentro da lista, letra por letra, usando um segundo índice.

Listas dentro de listas

```
>>> S=["maças", "peras", "kiwis"]
```

```
>>> print(S[0][0])
```

m

```
>>> print(S[0][1])
```

a

```
>>> print(S[1][1])
```

e

```
>>> print(S[2][2])
```

w

Impressão de uma lista de strings, letra a letra

```
L=["maçãs", "peras", "kiwis"]
```

```
for s in L:
```

```
    for letra in s:
```

```
        print(letra)
```

Isso nos leva a outra vantagem das listas em Python: listas dentro de listas.
Como
bônus, temos também que os elementos de uma lista não precisam ser do
mesmo tipo.

Listas com elementos de tipos diferentes

- Vejamos um exemplo onde teríamos uma lista de compras. O primeiro elemento seria o nome do produto; o segundo, a quantidade; e o terceiro, seu preço.

Listas com elementos de tipos diferentes

```
produto1 = [ "maçã", 10, 0.30]
```

```
produto2 = [ "pera", 5, 0.75]
```

```
produto3 = [ "kiwi", 4, 0.98]
```

Assim, produto1, produto2, produto3 seriam três listas com três elementos cada uma.

Observe que o primeiro elemento é do tipo string; o segundo, do tipo inteiro; e o terceiro, do tipo ponto flutuante (float)!

Listas de listas

```
produto1 = [ "maçã", 10, 0.30]
```

```
produto2 = [ "pera", 5, 0.75]
```

```
produto3 = [ "kiwi", 4, 0.98]
```

```
compras = [ produto1, produto2, produto3]
```

```
print(compras)
```

Agora temos uma lista chamada compras, também com três elementos, mas cada elemento é uma lista à parte.

Impressão das compras

```
produto1 = [ "maçã", 10, 0.30]
produto2 = [ "pera", 5, 0.75]
produto3 = [ "kiwi", 4, 0.98]
compras = [ produto1, produto2, produto3]
for e in compras:
    print("Produto: %s" % e[0])
    print("Quantidade: %d" % e[1])
    print("Preço: %5.2f" % e[2])
```

Da mesma forma, poderíamos ter acessado o preço do segundo elemento com
compras[1][2].

Criação e impressão da lista de compras

- Vejamos agora um programa completo, capaz de perguntar nome do produto, quantidade e preço e, no final, imprimir uma lista de compras completa.

Criação e impressão da lista de compras

```
compras = []

while True:
    produto = input("Produto: ")
    if produto == "fim":
        break
    quantidade = int(input("Quantidade: "))
    preço = float(input("Preço: "))
    compras.append([produto, quantidade, preço])

soma = 0.0

for e in compras:
    print("%20s x%5d %5.2f %6.2f" % (e[0], e[1], e[2], e[1] * e[2]))
    soma += e[1] * e[2]

print("Total: %7.2f" % soma)
```

Ordenação

- Até agora, os elementos de nossas listas apresentam a mesma ordem em que foram digitados, sem qualquer ordenação. Para ordenar uma lista, realizaremos uma operação semelhante à da pesquisa, mas trocando a ordem dos elementos quando necessário. Um algoritmo muito simples de ordenação é o “Bubble Sort”, ou método de bolhas, fácil de entender e aprender. Por ser lento, você não deve utilizá-lo com listas grandes.

Ordenação

- A ordenação pelo método de bolhas consiste em comparar dois elementos a cada vez. Se o valor do primeiro elemento for maior que o do segundo, eles trocarão de posição. Essa operação é então repetida para o próximo elemento até o fim a lista. O método de bolhas exige que percorramos a lista várias vezes. Por isso, utilizaremos um marcador para saber se chegamos ao fim da lista trocando ou não algum elemento. Esse método tem outra propriedade, que é posicionar o maior elemento na última posição da lista, ou seja, sua posição correta. Isso permite eliminar um elemento do fim da lista a cada passagem completa.

Ordenação

```
L=[7,4,3,12,8]
fim=5
while fim > 1:
    trocou=False
    x=0
    while x<(fim-1):
        if L[x] > L[x+1]:
            trocou=True
            temp=L[x]
            L[x]=L[x+1]
            L[x+1]=temp
        x+=1
    if not trocou:
        break
    fim-=1
for e in L:
    print(e)
```

Ordenação

- Utilizamos a variável fim para marcar a quantidade de elementos da lista. Precisamos marcar o fim ou a posição do último elemento porque no “Bubble Sort” não precisamos verificar o último elemento após uma passagem completa.

Ordenação

- Verificamos se $\text{fim} > 1$, pois como comparamos o elemento atual($L[x]$) com o seguinte($L[x+1]$), precisamos de, no mínimo, dois elementos.

Ordenação

- Utilizamos a variável trocou para indicar que não realizamos nenhuma troca.
- O valor de trocou será utilizado mais tarde para verificar se já temos a lista ordenada ou não.

Ordenação

- A variável x será utilizada como índice, começando da posição 0.

Ordenação

- Nossa condição do segundo while é especial, pois temos que garantir um próximo elemento para comparar com o atual. Isso faz com que a condição de saída desse while seja $x < (\text{fim} - 1)$, ou melhor, que x seja anterior ao último elemento.

Ordenação

- No if temos a comparação em si, onde x é nossa posição atual, e o próximo elemento é o de índice $x+1$.

Ordenação

- Como estamos ordenando em ordem crescente, desejamos que o próximo elemento sempre seja maior que o atual, dessa forma garantindo a ordenação da lista.
- Como comparamos apenas dois elementos de cada vez, temos que visitar a lista várias vezes, até que todos os elementos estejam em suas posições.
- Se a condição do if for verdadeira, esses elementos estão fora de ordem.
- Nesse caso, devemos inverter ou trocar a posição dos dois elementos.

Ordenação

- Dentro do if que efetuamos uma troca e, depois, utilizamos uma variável auxiliar ou temporária para trocar os dois valores de posição.

Ordenação

- Como cada variável só guarda um valor de cada vez, utilizaremos uma terceira variável temporária (temp) para armazenar o valor de uma delas durante a troca.

Ordenação

- A troca de valores entre duas variáveis é mostrada nas figuras abaixo

Ordenação

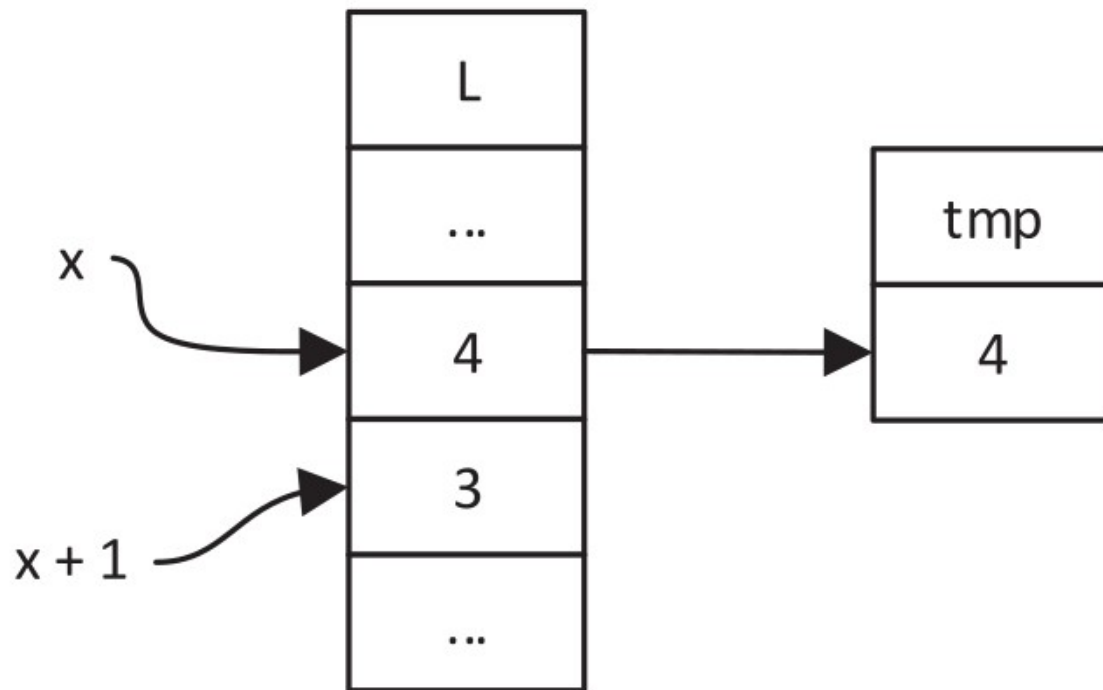


Figura 6.3 – Troca passo a passo. Primeira etapa.

Ordenação

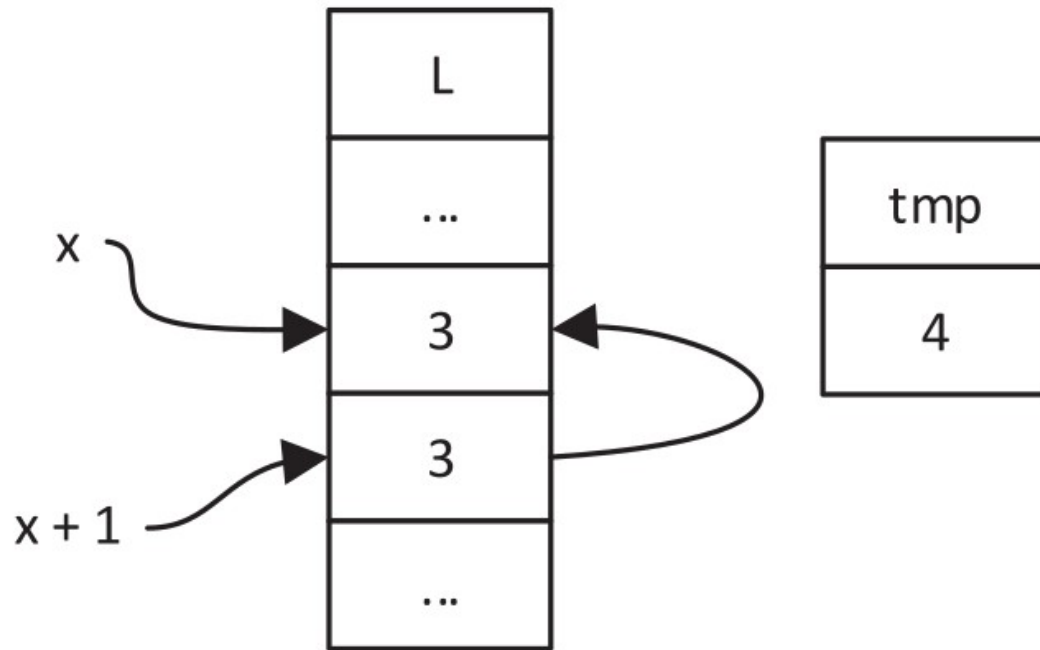


Figura 6.4 – Troca passo a passo. Segunda etapa.

Ordenação

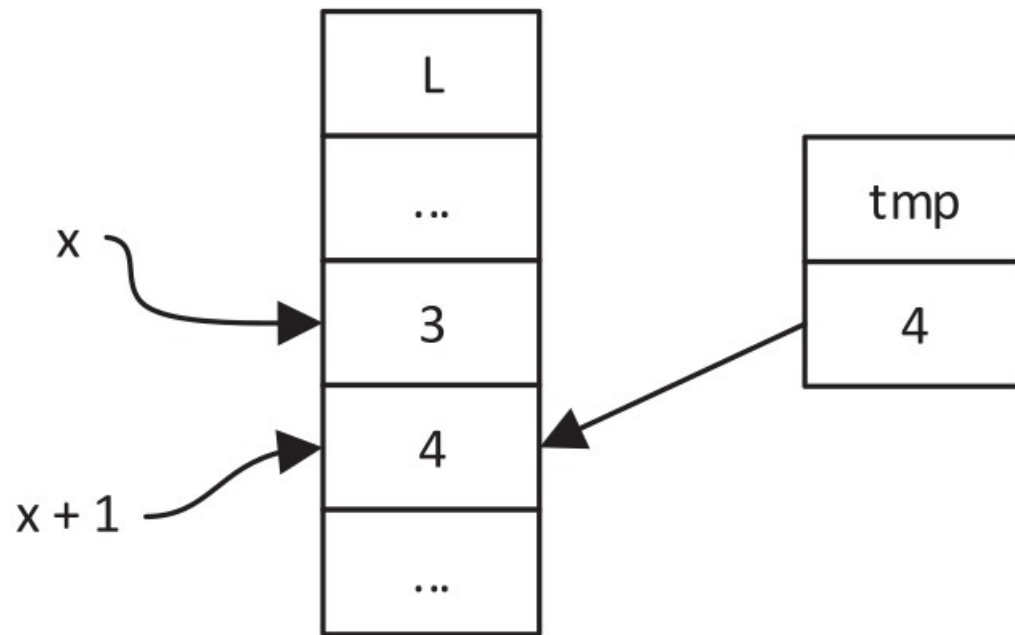


Figura 6.5 – Troca passo a passo. Terceira etapa.

Ordenação

- Você pode entender melhor o processo de troca de valores utilizando um problema do dia a dia: imagine que você tenha duas xícaras, uma com leite e outra com café. O problema consiste em passar o leite para a xícara que contém o café, e o café para a xícara que contém o leite, ou seja, você deve trocar o conteúdo das xícaras. Nesse caso, resolveríamos o problema utilizando uma terceira xícara para facilitar a operação. Fizemos o mesmo durante a ordenação, onde chamamos nossa terceira variável de temp.

Ordenação

- Quando a repetição iniciada no segundo while termina, o maior elemento está posicionado na última posição da lista, ou seja, em sua posição correta. No if not trocou, verificamos se algo foi trocado na repetição anterior. Se não trocamos nada de lugar, nossa lista está ordenada, e não precisamos executar a repetição outra vez, por isso o break.
- Caso contrário, como a última posição já está com o elemento correto, diminuiremos o valor de fim para que não precisemos mais verificá-lo.

Ordenação

- Vamos rastrear o algoritmo e observar como tudo isso funciona. Observe o rastreamento completo

[illegible]

Exercício 6.14

- O que acontece quando a lista já está ordenada? Rastreie o programa bubble sort, mas com a lista $L=[1,2,3,4,5]$.

Exercício 6.15

- O que acontece quando dois valores são iguais? Rastreie o programa bubble sort, mas com a lista $L=[3,3,1,5,4]$.

Exercício 6.16

- Modifique o programa bubble sort para ordenar a lista em ordem decrescente. $L=[1,2,3,4,5]$ deve ser ordenada como $L=[5,4,3,2,1]$.

Dicionários(Hash table)

- Dicionários consistem em uma estrutura de dados similar às listas, mas com propriedades de acesso diferentes. Um dicionário é composto por um conjunto de chaves e valores. O dicionário em si consiste em relacionar uma chave a um valor específico.

Dicionários

- Em Python, criamos dicionários utilizando chaves ({}).
- Cada elemento do dicionário é uma combinação de chave e valor.

Dicionários

- Vejamos um exemplo onde os preços de mercadorias sejam como os da tabela

Produto	Preço
Alface	R\$ 0,45
Batata	R\$ 1,20
Tomate	R\$ 2,30
Feijão	R\$ 1,50

A tabela pode ser vista como um dicionário, onde chave seria o produto; e valor, seu preço.

Criação de um dicionário

```
tabela = { "Alface": 0.45,  
"Batata": 1.20,  
"Tomate": 2.30,  
"Feijão": 1.50 }
```

Dicionários

- Para obter o preço da alface, digite no interpretador, depois de ter criado a tabela, `tabela["Alface"]`, onde `tabela` é o nome da variável do tipo dicionário, e “Alface” é nossa chave.
- O valor retornado é o mesmo que associamos na tabela, ou seja, 0,45.

Dicionários

- Diferentemente de listas, onde o índice é um número, dicionários utilizam suas chaves como índice.
- Quando atribuímos um valor a uma chave, duas coisas podem ocorrer:
 - Se a chave já existe: o valor associado é alterado para o novo valor.
 - Se a chave não existe: a nova chave será adicionada ao dicionário.

Funcionamento do dicionário

```
>>> tabela = { "Alface": 0.45,  
... "Batata": 1.20,  
... "Tomate": 2.30,  
... "Feijão": 1.50 }
```

```
>>> print(tabela["Tomate"])
```

2.3

```
>>> print(tabela)
```

```
{'Batata': 1.2, 'Alface': 0.45, 'Tomate': 2.3, 'Feijão': 1.5}
```

```
>>> tabela["Tomate"] = 2.50
```

```
>>> print(tabela["Tomate"])
```

2.5

```
>>> tabela["Cebola"] = 1.20
```

```
>>> print(tabela)
```

```
{'Batata': 1.2, 'Alface': 0.45, 'Tomate': 2.5, 'Cebola': 1.2, 'Feijão': 1.5}
```

Acesso a uma chave inexistente

```
>>> tabela = { "Alface": 0.45,  
... "Batata": 1.20,  
... "Tomate": 2.30,  
... "Feijão": 1.50 }  
>>> print(tabela["Manga"])  
Traceback (most recent call last):  
File "<stdin>", line 1, in <module>  
KeyError: 'Manga'
```

Se a chave não existir, uma exceção do tipo `KeyError` será ativada. Para verificar se

uma chave pertence ao dicionário, podemos usar o operador `in`

Verificação da existência de uma chave

```
>>> tabela = { "Alface": 0.45,  
... "Batata": 1.20,  
... "Tomate": 2.30,  
... "Feijão": 1.50 }
```

```
>>> print("Manga" in tabela)  
False
```

```
>>> print("Batata" in tabela)  
True
```

Obtenção de uma lista de chaves e valores

```
>>> tabela = { "Alface": 0.45,  
...  
"Batata": 1.20,  
...  
"Tomate": 2.30,  
...  
"Feijão": 1.50 }  
>>> print(tabela.keys())  
dict_keys(['Batata', 'Alface', 'Tomate', 'Feijão'])  
>>> print(tabela.values())  
dict_values([1.2, 0.45, 2.3, 1.5])
```

Observe que os métodos `keys()` e `values()` retornam geradores. Você pode utilizá-los diretamente dentro de um `for` ou transformá-los em lista usando a função `list`.

Obtenção do preço com um dicionário

```
tabela = { "Alface": 0.45,  
           "Batata": 1.20,  
           "Tomate": 2.30,  
           "Feijão": 1.50 }  
  
while True:  
    produto=input("Digite o nome do produto, fim para terminar:")  
    if produto == "fim":  
        break  
    if produto in tabela: ❶  
        print("Preço %5.2f" % tabela[produto]) ❷  
    else:  
        print("Produto não encontrado!")
```

Exclusão de uma associação do dicionário

```
>>> tabela = { "Alface": 0.45,  
... "Batata": 1.20,  
... "Tomate": 2.30,  
... "Feijão": 1.50 }  
>>> del tabela["Tomate"]  
>>> print(tabela)  
{ 'Batata': 1.2, 'Alface': 0.45, 'Feijão': 1.5 }
```

Dicionários com listas

- Em Python, podemos ter dicionários nos quais as chaves são associadas a listas ou mesmo a outros dicionários.

Dicionários com listas

- Imagine uma relação de estoque de mercadorias onde teríamos, além do preço, a quantidade em estoque:

```
estoque = { "tomate": [ 1000, 2.30],  
"alface": [ 500, 0.45],  
"batata": [ 2001, 1.20],  
"feijão": [ 100, 1.50] }
```

Nesse caso, o nome do produto é a chave, e a lista consiste nos valores associados,

uma lista por chave. O primeiro elemento da lista é a quantidade disponível; e o segundo, o preço do produto.

Dicionários com listas

- Uma aplicação seria processarmos uma lista de operações e calcular o preço total de venda, atualizando também a quantidade em estoque.

Exemplo de dicionário com estoque e operações de venda

```
estoque={ "tomate": [ 1000, 2.30],
          "alface": [ 500, 0.45],
          "batata": [ 2001, 1.20],
          "feijão": [ 100, 1.50] }

venda = [ ["tomate", 5], ["batata", 10], ["alface",5] ]
total = 0

print("Vendas:\n")

for operação in venda:
    produto, quantidade = operação ❶
    preço = estoque[produto][1] ❷
    custo = preço * quantidade
    print("%12s: %3d x %6.2f = %6.2f" %
          (produto, quantidade,preço,custo))
    estoque[produto][0] -= quantidade ❸
    total+=custo

print(" Custo total: %21.2f\n" % total)
print("Estoque:\n")

for chave, dados in estoque.items(): ❹
    print("Descrição: ", chave)
    print("Quantidade: ", dados[0])
    print("Preço: %6.2f\n" % dados[1])
```

Exercício 6.17

- Altere o programa da listagem 6.53 de forma a solicitar ao usuário o produto e a quantidade vendida. Verifique se o nome do produto digitado existe no dicionário, e só então efetue a baixa em estoque.

Exercício 6.18

- Escreva um programa que gere um dicionário, onde cada chave seja um caractere, e seu valor seja o número desse caractere encontrado em uma frase lida.
- Exemplo: O rato -> { "O":1, "r":1, "a":1, "t":1, "o":1}

Tuplas

- Tuplas podem ser vistas como listas em Python, com a grande diferença de serem imutáveis.

Tuplas

- Tuplas são criadas de forma semelhante às listas, mas utilizamos parênteses em vez de colchetes. Por exemplo:

```
>>> tupla = ("a", "b", "c")
```

```
>>> tupla
```

```
('a', 'b', 'c')
```


Tuplas

- Tuplas suportam a maior parte das operações de lista, como fatiamento e indexação:

```
>>> tupla = ("a", "b", "c")
```

```
>>> tupla
```

```
('a', 'b', 'c')
```

```
>>> tupla[0]
```

```
'a'
```

```
>>> tupla[2]
```

```
'c'
```

```
>>> tupla[1:]
```

```
('b', 'c')
```

```
>>> tupla * 2
```

```
('a', 'b', 'c', 'a', 'b', 'c')
```

```
>>> len(tupla)
```

```
6
```

Tuplas

- Mas tuplas não podem ter seus elementos alterados. Veja o que acontece se tentarmos alterar uma tupla:

```
>>> tupla[0]="A"
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: 'tuple' object does not support item assignment
```

Tuplas

- Várias funções utilizam ou geram tuplas em Python. Tuplas podem ser utilizadas com for:

```
>>> for elemento in tupla:
```

```
...
```

```
    print(elemento)
```

```
...
```

```
a
```

```
b
```

```
c
```

Tuplas

- Python também permite criar tuplas usando valores separados por vírgula, independente de usarmos parênteses:

```
>>> tupla = 100, 200, 300
```

```
>>> tupla
```

```
(100, 200, 300)
```

No caso, 100, 200 e 300 foram convertidos em uma tupla com três elementos. Esse tipo de operação é chamado de **empacotamento**.

Tuplas

- Tuplas também podem ser utilizadas para **desempacotar** valores, por exemplo:

```
>>> a, b = 10, 20
```

```
>>> a
```

```
10
```

```
>>> b
```

```
20
```

Onde o primeiro valor, 10, foi atribuído à primeira variável a e 20 a segunda, b. Esse tipo de construção é interessante para distribuímos o valor de uma tupla em várias variáveis.

Tuplas

- Esse tipo de construção é interessante para distribuímos o valor de uma tupla em várias variáveis.

```
>>> a, b = 10, 20
```

```
>>> a, b = b, a
```

```
>>> a
```

```
20
```

```
>>> b
```

```
10
```

Onde a tupla da esquerda foi usada para atribuir os valores à direita. Nesse caso, as atribuições: $a=b$ e $b = a$ foram realizadas imediatamente, sem precisar utilizarmos

Tuplas de um elemento

- A sintaxe do Python é um tanto especial quando precisamos criar tuplas com apenas um elemento. Como os valores são escritos entre parênteses, quando apenas um valor estiver presente, devemos acrescentar uma vírgula para indicar que o valor é uma tupla com apenas um elemento.

Tuplas de um elemento

- Veja o que acontece, usando e não usando a vírgula:

```
>>> t1 = (1)
```

```
>>> t1
```

```
1
```

```
>>> t2 = (1,)
```

```
>>> t2
```

```
(1,)
```

```
>>> t3 = 1,
```

```
>>> t3
```

```
(1,)
```

Veja que em t1 não utilizamos a vírgula, e o código foi interpretado como um número inteiro entre parênteses. Já em t2, utilizamos a vírgula, e nossa tupla foi corretamente construída. Em t3, criamos outra tupla, mas nesse caso nem pre-

Tuplas vazias

- Podemos também criar tuplas vazias, escrevendo apenas os parênteses:

```
>>> t4=()
```

```
>>> t4
```

```
()
```

```
>>> len(t4)
```

```
0
```

Tuplas a partir de listas

- Tuplas também podem ser criadas a partir de listas, utilizando-se a função tuple:

```
>>> L=[1,2,3]
```

```
>>> T=tuple(L)
```

```
>>> T
```

```
(1, 2, 3)
```

Tuplas concatenadas

- Embora não possamos alterar uma tupla depois de sua criação, podemos concatená-las, gerando novas tuplas:

```
>>> t1=(1,2,3)
```

```
>>> t2=(4,5,6)
```

```
>>> t1+t2
```

```
(1, 2, 3, 4, 5, 6)
```

Tuplas

- Observe que se uma tupla contiver uma lista ou outro objeto que pode ser alterado, este continuará a funcionar normalmente.

Tupla que contém uma lista

- Veja o exemplo de uma tupla que contém uma lista:

```
>>> tupla=("a", ["b", "c", "d"])
```

```
>>> tupla
```

```
('a', ['b', 'c', 'd'])
```

```
>>> len(tupla)
```

```
2
```

```
>>> tupla[1]
```

```
['b', 'c', 'd']
```

```
>>> tupla[1].append("e")
```

```
>>> tupla
```

```
('a', ['b', 'c', 'd', 'e'])
```

Neste caso, nada mudou na tupla em si, mas na lista que é seu segundo elemento.

Ou seja, a tupla não foi alterada, mas a lista que ela continha, sim.

Trabalhando com strings

- Vimos que podemos acessar strings como listas, mas também falamos que strings são imutáveis em Python.

Trabalhando com strings

```
>>> S="Alô mundo"
```

```
>>> print(S[0])
```

```
A
```

```
>>> S[0]="a"
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: 'str' object does not support item assignment
```

Trabalhando com strings

- Se quisermos trabalhar caractere a caractere com uma string, alterando seu valor, teremos que primeiramente transformá-la em uma lista

Convertendo uma string em lista

```
>>> L=list("Alô Mundo")
>>> L[0]="a"
>>> print(L)
['a', 'l', 'ô', ' ', 'M', 'u', 'n', 'd', 'o']
>>> s="".join(L)
>>> print(s)
alô Mundo
```

A função `list` transforma cada caractere da string em um elemento da lista retornada. Já o método `join` faz a operação inversa, transformando os elementos

Verificação parcial de strings

- Quando você precisar verificar se uma string começa ou termina com alguns caracteres, você pode usar os métodos `startswith` e `endswith`.
- Esses métodos verificam apenas os primeiros (`startswith`) ou os últimos (`endswith`) caracteres da string, retornando `True` caso sejam iguais ou `False` em caso contrário.

Verificação parcial de strings

```
>>> nome="João da Silva"
>>> nome.startswith("João")
True
>>> nome.startswith("joão")
False
>>> nome.endswith("Silva")
True
```

Veja que comparamos “João da Silva” com “joão” e obtivemos False. Esse é um detalhe pequeno, mas importante, pois startswith e endswith consideram letras maiúsculas e minúsculas como letras diferentes.

Conversão em maiúsculas e minúsculas

- Você pode resolver esse tipo de problema convertendo a string para maiúsculas ou minúsculas antes de realizar a comparação. O método `lower` retorna uma cópia da string com todos os caracteres minúsculos, e o método `upper` retorna uma cópia com todos os caracteres maiúsculos.

Conversão em maiúsculas e minúsculas

```
>>> s="O Rato roeu a roupa do Rei de Roma"
```

```
>>> s.lower()
```

```
'o rato roeu a roupa do rei de roma'
```

```
>>> s.upper()
```

```
'O RATO ROEU A ROUPA DO REI DE ROMA'
```

```
>>> s.lower().startswith("o rato")
```

```
True
```

```
>>> s.upper().startswith("O RATO")
```

```
True
```

Não se esqueça de comparar com uma string onde todos os caracteres são maiúsculos ou minúsculos, dependendo se você utilizou upper ou lower,

Pesquisa de palavras em uma string usando in

- Outra forma de verificar se uma palavra pertence a uma string é utilizando o operador in.

Pesquisa de palavras em uma string usando in

```
>>> s="Maria Amélia Souza"
```

```
>>> "Amélia" in s
```

```
True
```

```
>>> "Maria" in s
```

```
True
```

```
>>> "Souza" in s
```

```
True
```

```
>>> "a A" in s
```

```
True
```

```
>>> "amélia" in s
```

```
False
```

Pesquisa de palavras em uma string usando not in

- Você também pode testar se uma string não está contida em outra, utilizando not in

Pesquisa de palavras em uma string usando not in

```
>>> s="Todos os caminhos levam a Roma"
```

```
>>> "levam" not in s
```

```
False
```

```
>>> "Caminhos" not in s
```

```
True
```

```
>>> "AS" not in s
```

```
True
```

Veja que aqui também letras maiúsculas e minúsculas são diferentes.

Combinação de lower e upper com in e not in

- Você pode combinar lower e upper com in e not in para ignorar esse tipo de diferença

Combinação de lower e upper com in e not in

```
>>> s="João comprou um carro"
```

```
>>> "joão" in s.lower()
```

```
True
```

```
>>> "CARRO" in s.upper()
```

```
True
```

```
>>> "comprou" not in s.lower()
```

```
False
```

```
>>> "barco" not in s.lower()
```

```
True
```

O operador in também pode ser utilizado com listas normais, facilitando, assim, a pesquisa de elementos dentro de uma lista.

Contagem

- Se você precisar contar as ocorrências de uma letra ou palavra em uma string, utilize o método `count`.

Contagem de letra e palavras

```
>>> t="um tigre, dois tigres, três tigres"
```

```
>>> t.count("tigre")
```

```
3
```

```
>>> t.count("tigres")
```

```
2
```

```
>>> t.count("t")
```

```
4
```

```
>>> t.count("z")
```

```
0
```

Pesquisa de strings

- Para pesquisar se uma string está dentro de outra e obter a posição da primeira ocorrência, você pode utilizar o método find

Pesquisa de strings

```
>>> s="Alô mundo"
```

```
>>> s.find("mun")
```

```
4
```

```
>>> s.find("ok")
```

```
-1
```

Caso a string seja encontrada, você obterá um valor maior ou igual a zero, ou -1, em caso contrário. Observe que o valor retornado, quando maior ou igual a zero, é igual ao índice que pode ser utilizado para obter o primeiro caractere da string procurada.

Pesquisa de strings com rfind

- Se o objetivo for pesquisar, mas da direita para a esquerda, utilize o método rfind, que realiza essa tarefa

Pesquisa de strings com rfind

```
>>> s="Um dia de sol"
```

```
>>> s.rfind("d")
```

```
7
```

```
>>> s.find("d")
```

```
3
```

Pesquisa de strings, limitando o início ou o fim

- Tanto find quanto rfind suportam duas opções adicionais: início (start) e fim (end).
- Se você especificar início, a pesquisa começará a partir dessa posição. Se você especificar o fim, a pesquisa utilizará essa posição como último caractere a considerar na pesquisa.

Pesquisa de strings, limitando o início ou o fim

- `>>> s="um tigre, dois tigres, três tigres"`
- `>>> s.find("tigres")`
- `15`
- `>>> s.rfind("tigres")`
- `28`
- `>>> s.find("tigres",7) #início=7`
- `15`
- `>>> s.find("tigres",30) #início=30`
- `-1`
- `>>> s.find("tigres",0,10) #início=0 fim=10`
- `-1`

Pesquisa de todas as ocorrências

- Podemos usar o valor retornado por `find` e `rfind` para achar todas as ocorrências da string.

Pesquisa de todas as ocorrências

```
s="um tigre, dois tigres, três tigres"
```

```
p=0
```

```
while(p>-1):
```

```
    p=s.find("tigre", p)
```

```
    if p>=0:
```

```
        print("Posição: %d" % p)
```

```
        p+=1
```

Posição: 3

Posição: 15

Posição: 28

Os métodos `index` e `rindex` são bem parecidos com `find` e `rfind`, respectivamente. A maior diferença é que se a substring não for encontrada, `index` e `rindex` lançam uma exceção do tipo `ValueError`.

Exercício 7.1

- Escreva um programa que leia duas strings. Verifique se a segunda ocorre dentro da primeira e imprima a posição de início.
 - 1ª string: AABBEFAATT
 - 2ª string: BE
- Resultado: BE encontrado na posição 3 de AABBEFAATT

Exercício 7.2

- Escreva um programa que leia duas strings e gere uma terceira com os caracteres comuns às duas strings lidas.
 - 1ª string: AAACCTBF
 - 2ª string: CBT
- Resultado: CBT
- A ordem dos caracteres da string gerada não é importante, mas deve conter todas as letras comuns a ambas.

Exercício 7.3

- Escreva um programa que leia duas strings e gere uma terceira apenas com os caracteres que aparecem em uma delas.
 - 1ª string: CTA
 - 2ª string: ABC
 - 3ª string: BT
- A ordem dos caracteres da terceira string não é importante.

Exercício 7.4

- Escreva um programa que leia uma string e imprima quantas vezes cada caractere aparece nessa string.
 - String: TTAAC
- Resultado:

T: 2x

A: 2x

C: 1x

Exercício 7.5

- Escreva um programa que leia duas strings e gere uma terceira, na qual os caracteres da segunda foram retirados da primeira.
 - 1ª string: AATTGGAA
 - 2ª string: TG
 - 3ª string: AAAA

Exercício 7.6

- Escreva um programa que leia três strings. Imprima o resultado da substituição na primeira, dos caracteres da segunda pelos da terceira.
 - 1ª string: AATTCGAA
 - 2ª string: TG
 - 3ª string: AC
- Resultado: AAAACCAA

Posicionamento de strings

- Python também traz métodos que ajudam a apresentar strings de formas mais interessantes.
- Vejamos o método `center`, que centraliza a string em um número de posições passado como parâmetro, preenchendo com espaços à direita e à esquerda até que a string esteja centralizada

Centralização de texto em uma string

```
>>> s="tigre"
```

```
>>> print("X"+s.center(10)+"X")
```

```
X tigre X
```

```
>>> print("X"+s.center(10,".")+"X")
```

```
X..tigre...X
```

Se, além do tamanho, você também passar o caractere de preenchimento, este será utilizado no lugar de espaços em branco.

Preenchimento de strings com espaços

```
>>> s="tigre"  
>>> s.ljust(20)  
'tigre          '  
>>> s.rjust(20)  
'          tigre'  
>>> s.ljust(20,".")  
'tigre.....'  
>>> s.rjust(20,"-")  
'-----tigre'
```

Essas funções são úteis quando precisamos criar relatórios ou simplesmente alinhar a saída dos programas.

Quebra ou separação de strings

- O método `split` quebra uma string a partir de um caractere passado como parâmetro, retornando uma lista com as substrings já separadas.

Separação de strings

```
>>> s="um tigre, dois tigres, três tigres"
```

```
>>> s.split(",")
```

```
['um tigre', ' dois tigres', ' três tigres']
```

```
>>> s.split(" ")
```

```
['um', 'tigre,', 'dois', 'tigres,', 'três', 'tigres']
```

```
>>> s.split()
```

```
['um', 'tigre,', 'dois', 'tigres,', 'três', 'tigres']
```

Observe que o caractere que utilizamos para dividir a string não é retornado na

lista, ou seja, ele é utilizado para separar a string e depois descartado.

Quebra de strings de várias linhas

- Se você deseja separar uma string, com várias linhas de texto, você pode utilizar o método `splitlines`

```
>>> m="Uma linha\noutra linha\ne mais uma\n"
```

```
>>> m.splitlines()
```

```
['Uma linha', 'outra linha', 'e mais uma']
```

Substituição de strings

```
>>> s="um tigre, dois tigres, três tigres"
>>> s.replace("tigre", "gato")
'um gato, dois gatos, três gatos'
>>> s.replace("tigre", "gato", 1)
'um gato, dois tigres, três tigres'
>>> s.replace("tigre", "gato", 2)
'um gato, dois gatos, três tigres'
>>> s.replace("tigre", "")
'um , dois s, três s'
>>> s.replace("", "-")
'-u-m- -t-i-g-r-e-, -d-o-i-s- -t-i-g-r-e-s-, -t-r-ê-s- -t-i-g-r-e-s-'
```

Se você passar uma string vazia no segundo parâmetro, o trecho será apagado.

Se o primeiro parâmetro for uma string vazia, o segundo será inserido antes de

Remoção de espaços em branco

- O método `strip` é utilizado para remover espaços em branco do início ou fim da string. Já os métodos `lstrip` e `rstrip` removem apenas os caracteres em branco à esquerda ou à direita, respectivamente

Remoção de espaços em branco com strip, lstrip e rstrip

```
>>> t="  Olá  "
```

```
>>> t.strip()
```

```
'Olá'
```

```
>>> t.lstrip()
```

```
'Olá  '
```

```
>>> t.rstrip()
```

```
'  Olá'
```

Remoção de caracteres com strip, lstrip e rstrip

- Se você passar um parâmetro tanto para strip quanto para lstrip ou rstrip, este será utilizado como caractere a remover

Remoção de caracteres com strip, lstrip e rstrip

```
>>> s="...///Olá///..."
```

```
>>> s.lstrip(".")
```

```
'///Olá///...'
```

```
>>> s.rstrip(".")
```

```
'...///Olá//'
```

```
>>> s.strip(".")
```

```
'///Olá//'
```

```
>>> s.strip("./")
```

```
'Olá'
```

Validação por tipo de conteúdo

- Strings em Python podem ter seu conteúdo analisado e verificado utilizando métodos especiais.
- Esses métodos verificam se todos os caracteres são letras, números ou uma combinação deles.

Validação de strings por seu conteúdo

```
>>> s="125"
```

```
>>> p="alô mundo"
```

```
>>> s.isalnum()
```

```
True
```

```
>>> p.isalnum()
```

```
False
```

```
>>> s.isalpha()
```

```
False
```

```
>>> p.isalpha()
```

```
False
```


Validação de strings por seu conteúdo

- O método `isalnum` retorna verdadeiro se a string não estiver vazia, e se todos os seus caracteres são letras e/ou números.
- Se a string contiver outros tipos de caracteres, como espaços, vírgula, exclamação, interrogação ou caracteres de controle, retorna `False`.

Validação de strings por seu conteúdo

- Já isalpha é mais restritivo, retornando verdadeiro apenas se todos os caracteres forem letras, incluindo vogais acentuadas. Retorna falso se algum outro tipo de caractere for encontrado na string ou se estiver vazia.

Validação de strings com números

- O método `isdigit` verifica se o valor consiste em números, retornando `True` se a string não estiver vazia e contiver apenas números.
- Se a string contiver espaços, pontos, vírgulas ou sinais (+ ou -), retorna falso

Validação de strings com números

- `>>> "771".isdigit()`
- `True`
- `>>> "10.4".isdigit()`
- `False`
- `>>> "+10".isdigit()`
- `False`
- `>>> "-5".isdigit()`
- `False`

Diferenciação de `isnumeric` de `isdigit`

- Os métodos `isdigit` e `isnumeric` são parecidos, e diferenciá-los envolve um conhecimento de Unicode mais aprofundado. `isdigit` retorna `True` para caracteres definidos como dígitos numéricos em Unicode, indo além de nossos 0 a 9, como, por exemplo um 9 tibetano (`\u0f29`).

Diferenciação de `isnumeric` de `isdigit`

- `isnumeric` é mais abrangente, incluindo dígitos e representações numéricas como frações, por exemplo, `1/3` (`\u2153`).

Diferenciação de isnumeric de isdigit

```
>>> umterço="\u2153"
```

```
>>> novetibetano="\u0f29"
```

```
>>> umterço.isdigit()
```

```
False
```

```
>>> umterço.isnumeric()
```

```
True
```

```
>>> novetibetano.isdigit()
```

```
True
```

```
>>> novetibetano.isnumeric()
```

```
True
```

Verificação de maiúsculas e minúsculas

- Podemos também verificar se todos os caracteres de uma string são letras maiúsculas ou minúsculas usando `isupper` e `islower`, respectivamente

Verificação de maiúsculas e minúsculas

```
>>> s="ABC"
```

```
>>> p="abc"
```

```
>>> e="aBc"
```

```
>>> s.isupper()
```

```
True
```

```
>>> s.islower()
```

```
False
```

```
>>> p.isupper()
```

```
False
```

```
>>> p.islower()
```

```
True
```

```
>>> e.isupper()
```

```
False
```

```
>>> e.islower()
```

```
False
```

Formatação de strings

- A versão 3 do Python introduziu uma nova forma de representar máscaras em strings. Essa nova forma é mais poderosa que as tradicionais máscaras que utilizamos, combinando %d, %s, %f.

Formatação de strings com o método format

```
>>> "{0} {1}".format("Alô", "Mundo")
```

```
'Alô Mundo'
```

```
>>> "{0} x {1} R${2}".format(5, "maçã", "1.20")
```

```
'5 x maçã R$1.20'
```

O número entre colchetes é uma referência aos parâmetros passados ao método

format, onde 0 é o primeiro parâmetro; 1, o segundo; e assim por diante, como os índices de uma lista.

Uso do mesmo parâmetro mais de uma vez

- Uma das vantagens da nova sintaxe é poder utilizar o mesmo parâmetro várias vezes na string

```
>>> "{0} {1} {0}".format("-", "x")
```

```
'- x -'
```

Alteração da ordem de utilização dos parâmetros

- Isso também permite a completa reordenação da mensagem, como imprimir os parâmetros em outra ordem

```
>>> "{1} {0}".format("primeiro", "segundo")
```

```
'segundo primeiro'
```

Limitação do tamanho de impressão dos parâmetros

- A nova sintaxe permite também especificar a largura de cada valor, utilizando o símbolo de dois pontos (:) após a posição do parâmetro, como 0:10, que significa: substitua o primeiro parâmetro com uma largura de 10 caracteres.

Limitação do tamanho de impressão dos parâmetros

- Se o primeiro parâmetro for menor que o tamanho informado, espaços serão utilizados para completar as posições que faltam. Se o parâmetro for maior que o tamanho especificado, ele será impresso em sua totalidade, ocupando mais espaço que o inicialmente especificado.

Limitação do tamanho de impressão dos parâmetros

```
>>> "{0:10} {1}".format("123", "456")
```

```
'123      456'
```

```
>>> "X{0:10}X".format("123")
```

```
'X123      X'
```

```
>>> "X{0:10}X".format("123456789012345")
```

```
'X123456789012345X'
```


Especificação de espaços à esquerda ou à direita

- Podemos também especificar se queremos os espaços adicionais à esquerda ou à direita do valor, utilizando os símbolos de maior ($>$) e menor ($<$) logo depois dos dois pontos

Especificação de espaços à esquerda ou à direita

```
>>> "X{0:<10}X".format("123")
```

```
'X123      X'
```

```
>>> "X{0:>10}X".format("123")
```

```
'X      123X'
```

Centralização

- Se quisermos o valor entre os espaços, de forma a centralizá-lo, podemos utilizar o circunflexo (^)

Centralização

- `>>> "X{0:^10}X".format("123")`
- `'X 123 X'`

Especificação de espaços à esquerda ou à direita

- Se quisermos outro caractere no lugar de espaços, podemos especificá-lo logo após os dois pontos

Especificação de espaços à esquerda ou à direita

```
>>> "X{0:.<10}X".format("123")
```

```
'X123.....X'
```

```
>>> "X{0:!!>10}X".format("123")
```

```
'X!!!!!!!!123X'
```

```
>>> "X{0:*^10}X".format("123")
```

```
'X***123***X'
```

Máscaras com elementos de uma lista

- Se o parâmetro for uma lista, podemos especificar o índice do elemento a substituir, dentro da máscara

Máscaras com elementos de uma lista

- `>>> "{0[0]} {0[1]}".format(["123", "456"])`
- `'123 456'`

Máscaras com elementos de um dicionário

- O mesmo é válido para dicionários

Máscaras com elementos de um dicionário

```
>>> "{0[nome]} {0[telefone]}".format({ "telefone":  
572, "nome":"Maria"})
```

```
'Maria 572'
```

Observe que dentro da string escrevemos nome e telefone entre colchetes, mas sem aspas, como normalmente faríamos ao utilizar dicionários. Essa sintaxe é especial para o método format.

Formatação de números

- A nova sintaxe também permite a formatação de números. Por exemplo, se especificarmos o tamanho a imprimir com um zero à esquerda, o valor será impresso com a largura determinada e com zeros à esquerda completando o tamanho

Zeros à esquerda

```
>>> "{0:05}".format(5)
```

```
'00005'
```

Preenchimento com outros caracteres

- Podemos também utilizar outro caractere, diferente de 0, mas, nesse caso, devemos escrever o caractere à esquerda do símbolo de igualdade

Preenchimento com outros caracteres

```
>>> "{0:*=7}".format(32)
```

```
'*****32'
```

Combinação de vários códigos de formatação

- Podemos também especificar o alinhamento dos números que estamos imprimindo usando $<$, $>$ e $^$

Combinação de vários códigos de formatação

```
>>> "{0:*^10}".format(123)
```

```
'***123***'
```

```
>>> "{0:*<10}".format(123)
```

```
'123*****'
```

```
>>> "{0:*>10}".format(123)
```

```
'*****123'
```


Separação de milhares

- Podemos também utilizar uma vírgula para solicitar o agrupamento por milhar, e o ponto para indicar a precisão de números decimais, ou melhor, a quantidade de casas após a vírgula.

Separação de milhares

```
>>> "{0:10,}".format(7532)
```

```
'    7,532'
```

```
>>> "{0:10.5f}".format(1500.31)
```

```
'1500.31000'
```

```
>>> "{0:10,.5f}".format(1500.31)
```

```
'1,500.31000'
```

Impressão de sinais de positivo e negativo

- A nova sintaxe também permite forçar a impressão de sinais ou apenas reservar espaço para uma impressão eventual

Impressão de sinais de positivo e negativo

- `>>> "{0:+10} {1:-10}".format(5,-6)`
- `' +5 -6'`
- `>>> "{0:-10} {1: 10}".format(5,-6)`
- `' 5 -6'`
- `>>> "{0: 10} {1:+10}".format(5,-6)`
- `' 5 -6'`

Formatação de inteiros

- Quando trabalhamos com formatos numéricos, devemos indicar com uma letra o formato que deve ser adotado para a impressão. Essa letra informa como devemos exibir um número.

Formatação de inteiros

Código	Descrição	Exemplo (45)
b	Binário	101101
c	Caractere	-
d	Base 10	45
n	Base 10 local	45
o	Octal	55
x	Hexadecimal com letras minúsculas	2d
X	Hexadecimal com letras maiúsculas	2D

Formatação de inteiros

- O formato b imprime o número utilizando o sistema binário, ou seja, de base 2, com apenas 0 e 1 como dígitos.
- O formato o imprime o número utilizando o sistema octal, ou seja, de base 8, com dígitos de 0 a 7.
- Já o formato c imprime o número convertendo-o em caractere, utilizando a tabela Unicode.
- Tanto o formato x quando o X imprimem os números utilizando o sistema hexadecimal, base 16. A diferença é que x utiliza letras minúsculas, e X, maiúsculas.

Formatação de inteiros

```
>>> "{:b}".format(5678)
```

```
'1011000101110'
```

```
>>> "{:c}".format(65)
```

```
'A'
```

```
>>> "{:o}".format(5678)
```

```
'13056'
```

```
>>> "{:x}".format(5678)
```

```
'162e'
```

```
>>> "{:X}".format(5678)
```

```
'162E'
```


O formato d e o formato n

- Os formatos d e n são parecidos. O formato d é semelhante ao que utilizamos ao formatar os números com %d. A diferença entre o formato d e o n é que o n leva em consideração as configurações regionais da máquina do usuário.

O formato d e o formato n

```
>>> "{:d}".format(5678)
```

```
'5678'
```

```
>>> "{:n}".format(5678)
```

```
'5678'
```

```
>>> import locale
```

```
>>> locale.setlocale(locale.LC_ALL,"pt_BR.utf-8")
```

```
'pt_BR.utf-8'
```

```
>>> "{:n}".format(5678)
```

```
'5.678'
```

Observe que, antes de configurarmos a máquina para o português do Brasil, o resultado de d e n eram iguais. Após a configuração regional, o formato n passou a exibir os números utilizando pontos para separar os milhares. Se você utiliza Windows, modifique "pt_BR.utf-8" para "Portuguese_Brazil"

Formatação de números decimais

- Para números decimais, temos também vários códigos. O código `f` já conhecemos e funciona de forma semelhante ao que utilizamos em `%f`. O formato `n` utiliza as configurações regionais para imprimir o número. Em português, essa configuração utiliza o ponto como separador de milhar e a vírgula como separador decimal, produzindo números mais fáceis de entender.

Formatação de números decimais

```
>>> "{:f}".format(1579.543)
```

```
'1579.543000'
```

```
>>> "{:n}".format(1579.543)
```

```
'1579.54'
```

```
>>> import locale
```

```
>>> locale.setlocale(locale.LC_ALL,"pt_BR.utf-8")
```

```
'pt_BR.utf-8'
```

```
>>> "{:n}".format(1579.543)
```

```
'1.579,54'
```

Formatação de números decimais

Código	Descrição	Exemplo (1.345)
e	Notação científica com e minúsculo	1.345000e+00
E	Notação científica com e maiúsculo	1.345000E+00
f	Decimal	1.345000
g	Genérico	1.345
G	Genérico	1.345
n	Local	1,345
%	Percentual	134.500000%

Notação científica

- Os formatos e e E imprimem o número utilizando notação científica. Isso quer dizer que a parte decimal vai ser substituída por um expoente.

Notação científica

- Por exemplo: 1004.5 em notação científica será impresso 1.004500e+03. Para entender esse formato, devemos entender suas partes. O número à esquerda de e é o que chamamos de mantissa, e o número à direita, o expoente. A base é sempre 10 e, para recompor o número, multiplicamos a mantissa pela base 10 elevada ao expoente.

Notação científica

- No caso de 1.004500e+03, teríamos:
- $1.004500 \times 10^3 = 1.004500 \times 1000 = 1004.5$

Notação científica

- A vantagem de utilizar notação científica é poder representar números muito grandes, ou muito pequenos, em pouco espaço. A diferença entre o formato e e o E é que queremos exibir o e que separa a mantissa do expoente, ou seja, um e minúsculo ou um E maiúsculo.

Formatação de números decimais

- Os formatos g e G são chamados de genéricos, pois, dependendo do número, eles são exibidos como no formato f ou como no e ou E.
- O tipo % simplesmente multiplica o valor por 100 antes de imprimi-lo, assim, 0,05 é impresso como 5%.

Formatação de números decimais

```
>>> "{:8e}".format(3.141592653589793)
```

```
' 3.141593e+00'
```

```
>>> "{:8E}".format(3.141592653589793)
```

```
' 3.141593E+00'
```

```
>>> "{:8g}".format(3.141592653589793)
```

```
' 3.14159'
```

```
>>> "{:8G}".format(3.141592653589793)
```

```
' 3.14159'
```

```
>>> "{:8g}".format(3.14)
```

```
' 3.14'
```

```
>>> "{:8G}".format(3.14)
```

```
' 3.14'
```

```
>>> "{:5.2%}".format(0.05)
```

```
'5.00%'
```

Jogo da forca

- Vejamos um jogo muito simples, mas que ajuda a trabalhar com strings.

```
palavra = input("Digite a palavra secreta:").lower().strip()
for x in range(100):
    print()
digitadas = []
acertos = []
erros = 0
while True:
    senha=""
    for letra in palavra:
        senha +=letra if letra in acertos else "."
    print(senha)
    if senha == palavra:
        print("Você acertou!")
        break
    tentativa = input("\nDigite uma letra:").lower().strip()
    if tentativa in digitadas:
        print("Você já tentou esta letra!")
        continue
    else:
        digitadas += tentativa
        if tentativa in palavra:
            acertos += tentativa
        else:
            erros += 1
            print("Você errou!")
            print("X==:==\nX : ")
            print("X |0 " if erros >= 1 else "X")
            linha2=""
```

```
if erros == 2:
    linha2 = " | "
elif erros == 3:
    linha2 = " \| "
elif erros >= 4:
    linha2 = " \|| "
    print("X%s" % linha2)
    linha3=""
if erros == 5:
    linha3+=" / "
elif erros>=6:
    linha3+=" / \ "
    print("X%s" % linha3)
    print("X\n=====")
if erros == 6:
    print("Enforcado!")
    break
```

Exercício 7.7

- Modifique o programa de forma a escrever a palavra secreta caso o jogador perca.

Exercício 7.8

- Modifique o jogo da forca de forma a utilizar uma lista de palavras.
- No início, pergunte um número e calcule o índice da palavra a utilizar pela fórmula: $\text{índice} = (\text{número} * 776) \% \text{len}(\text{lista_de_palavras})$.

Exercício 7.9

- Modifique o programa para utilizar listas de strings para desenhar o boneco da forca. Você pode utilizar uma lista para cada linha e organizá-las em uma lista de listas. Em vez de controlar quando imprimir cada parte, desenhe nessas listas, substituindo o elemento a desenhar.
- Exemplo:

```
>>> linha = list("X-----")
```

```
>>> linha
```

```
['X', '-', '-', '-', '-', '-', '-']
```

```
>>> linha[6] = "|"
```

```
>>> linha
```

```
['X', '-', '-', '-', '-', '-', '|']
```

```
>>> "".join(linha)
```

```
'X-----|'
```

Exercício 7.10

- Escreva um jogo da velha para dois jogadores. O jogo deve perguntar onde você quer jogar e alternar entre os jogadores. A cada jogada, verifique se a posição está livre. Verifique também quando um jogador venceu a partida. Um jogo da velha pode ser visto como uma lista de 3 elementos, onde cada elemento é outra lista, também com três elementos.

Exercício 7.10

- Exemplo do jogo:

```
X | O |  
---+---+---  
  | X | X  
---+---+---  
  |   | O
```

Exercício 7.10

- Onde cada posição pode ser vista como um número. Confira abaixo um exemplo das posições mapeadas para a mesma posição de seu teclado numérico.

7 | 8 | 9

---+---+---

4 | 5 | 6

---+---+---

1 | 2 | 3

Funções

- Podemos definir nossas próprias funções em Python. Sabemos como usar várias funções, como len, int, float, print e input. Neste capítulo, veremos como declarar novas funções e utilizá-las em programas.

Funções

- Para definir uma nova função, utilizaremos a instrução `def`.

Funções

- Vejamos como declarar uma função de soma que recebe dois números como parâmetros e os imprime na tela

Definição de uma nova função

```
def soma(a,b):  
    print(a+b)
```


Definição do retorno de um valor

```
def soma(a,b):  
    return(a+b)
```

Retornando se valor é par ou não

```
def épar(x):  
    return(x%2==0)
```

Reutilização da função épar em outra função

```
def épar(x):  
    return(x%2==0)  
def par_ou_ímpar(x):  
    if épar(x):  
        return "par"  
    else:  
        return "ímpar"
```

Exercício 8.1

Escreva uma função que retorne o maior de dois números.

Valores esperados:

- `máximo(5,6) == 6`
- `máximo(2,1) == 2`
- `máximo(7,7) == 7`

Exercício 8.2

Escreva uma função que receba dois números e retorne True se o primeiro número for múltiplo do segundo.

Valores esperados:

- `múltiplo(8,4) == True`
- `múltiplo(7,3) == False`
- `múltiplo(5,5) == True`

Exercício 8.3

Escreva uma função que receba o lado (l) de um quadrado e retorne sua área ($A = l^2$).

Valores esperados:

- `área_quadrado(4) == 16`
- `área_quadrado(9) == 81`

Exercício 8.4

Escreva uma função que receba a base e a altura de um triângulo e retorne sua área ($A = (\text{base} \times \text{altura})/2$).

Valores esperados:

- `área_triângulo(6, 9) == 27`
- `área_triângulo(5, 8) == 20`

Pesquisa em uma lista

```
def pesquise(lista, valor):  
    for x,e in enumerate(lista):  
        if e == valor:  
            return x  
    return None
```


Cálculo da média de uma lista

```
def soma(L):
```

```
    total=0
```

```
    for e in L:
```

```
        total+=e
```

```
    return total
```

```
def media(L):
```

```
    return(soma(L)/len(L))
```

Para calcular a média, definimos outra função chamada soma. Dessa forma, teremos duas funções, uma para calcular a média e outra para calcular a soma da lista.

Funções

- Definir as funções dessa forma é mais interessante, **pois uma função deve resolver apenas um problema.**

Soma e cálculo da média de uma lista

- Poderíamos também ter definido a função média como abaixo:

```
def média(L):  
    total=0  
    for e in L:  
        total+=e  
    return total/len(L)
```

Funções

- Qual das duas formas escolher depende do tipo de problema que se quer resolver.
- Se você não precisa do valor da soma dos elementos de uma lista em nenhuma parte do programa, a segunda forma é interessante.
- A primeira (soma e média definidas em duas funções separadas) é mais interessante em longo prazo e é também uma boa prática de programação.
- Conforme formos criando funções, podemos armazená-las para uso em outros programas.

Funções

- Por enquanto, tente fixar duas regras:
 - Uma função deve resolver apenas um problema
 - Quanto mais genérica for sua solução, melhor ela será em longo prazo.

Como não escrever uma função

```
def soma(L):  
    total=0  
    x = 0  
    while x<5:  
        total+=L[x]  
        x+=1  
    return total
```

A forma como definimos a função soma não é genérica, ou melhor, só funciona em casos onde devemos somar listas com cinco elementos.

Funções

- Um problema clássico de programação é o cálculo do fatorial.
- O fatorial de um número é utilizado em estatística para calcular o número de combinações e permutações de conjuntos.

Cálculo do fatorial

- Para calcular o fatorial, multiplicamos o número por todos os números que o precedem até chegarmos em 1
- Por exemplo:
 - fatorial de 3: $3 \times 2 \times 1 = 6$.
 - fatorial de 4: $4 \times 3 \times 2 \times 1 = 24$

Um caso especial é o fatorial de 0 que é definido como 1.

Cálculo do fatorial

```
def fatorial(n):
```

```
    fat = 1
```

```
    while n>1:
```

```
        fat*=n
```

```
        n-=1
```

```
    return fat
```

Se você rastrear essa função, verá que calculamos o fatorial multiplicando o valor

de n pelo valor anterior (fat) e que começamos do maior número até chegarmos em 1. Observe que a forma que definimos a função satisfaz o caso especial do fatorial de zero.

Outra forma de calcular o fatorial

```
def fatorial(n):
```

```
    fat=1
```

```
    x=1
```

```
    while x<=n:
```

```
        fat*=x
```

```
        x+=1
```

```
    return fat
```

Existem várias formas de resolver o problema, e todas podem estar corretas. Para decidir se nossa implementação está correta, temos que observar os valores retornados e compará-los aos valores de referência.

Exercício 8.5

- Reescreva a função da listagem 8.5(pesquisa em lista) de forma a utilizar os métodos de pesquisa em lista, vistos no capítulo 7.

Exercício 8.6

- Reescreva o programa da listagem 8.8(cálculo do somatório de uma lista) de forma a utilizar for em vez de while.

Dica

- Python tem funções para calcular a soma, o máximo e o mínimo de uma lista.

```
>>> L=[5,7,8]
```

```
>>> sum(L)
```

```
20
```

```
>>> max(L)
```

```
8
```

```
>>> min(L)
```

```
5
```

Variáveis locais e globais

- Quando usamos funções, começamos a trabalhar com variáveis internas ou locais e com variáveis externas ou globais.
- A diferença entre elas é a visibilidade ou escopo.

Variáveis locais e globais

- Uma variável local de uma função existe apenas dentro dela, sendo normalmente inicializada a cada chamada.
- Assim, não podemos acessar o valor de uma variável local fora da função que a criou e, por isso, passamos parâmetros e retornamos valores nas funções, de forma a possibilitar a troca de dados no programa.

Variáveis locais e globais

- Uma variável global é definida fora de uma função, pode ser vista por todas as funções do módulo e por todos os módulos que importam o módulo que a definiu.

Variáveis locais e globais

```
EMPRESA="Unidos Venceremos Ltda"
```

```
def imprime_cabecalho():
```

```
    print(EMPRESA)
```

```
    print("-" * len(EMPRESA))
```

A função `imprime_cabecalho` não recebe parâmetros nem retorna valores. Ela simplesmente imprime o nome da empresa e traços abaixo dele. Observe que utilizamos a variável `EMPRESA` definida fora da função. Nesse caso, `EMPRESA` é uma variável global, podendo ser acessada em qualquer função.

Variáveis locais e globais

- Variáveis globais devem ser utilizadas o mínimo possível em seus programas, pois dificultam a leitura e violam o encapsulamento da função.

Variáveis locais e globais

- A dificuldade da leitura é em procurar pela definição e conteúdos fora da função em si, que podem mudar entre diferentes chamadas.
- Além disso, uma variável global pode ser alterada por qualquer função, tornando a tarefa de saber quem altera seu valor realmente mais trabalhosa.

Variáveis locais e globais

- O encapsulamento é comprometido porque a função depende de uma variável externa, ou seja, que não é declarada dentro da função nem recebida como parâmetro.

Variáveis locais e globais

- Embora devamos utilizar variáveis globais com cuidado, isso não significa que elas não tenham uso ou que possam simplesmente ser classificadas como má prática.
- Um bom uso de variáveis globais é guardar valores constantes e que devem ser acessíveis a todas as funções do programa, como o nome da empresa.

Variáveis locais e globais

```
a=5 ❶  
  
def muda_e_imprime():  
    a=7 ❷  
    print("A dentro da função: %d" % a)  
print("a antes de mudar: %d" % a) ❸  
muda_e_imprime() ❹  
print("a depois de mudar: %d" % a) ❺
```

Em ❶, criamos a variável global a. Em ❷, temos a variável local da função, também chamada a, recebendo 7. Em ❸ e ❺, imprimimos o valor da variável global a e, em ❹, o valor da variável local a. Para o computador, essas variáveis são completamente diferentes, embora tenham o mesmo nome. Em ❹, podemos acessar seu conteúdo porque realizamos a impressão dentro da função. Da mesma forma, a variável a que imprimimos é a variável local, valendo 7. Essa variável local deixa de existir no final da função e explica porque não alteramos a variável global a.

Variáveis locais e globais

- Se quisermos modificar uma variável global dentro de uma função, devemos informar que estamos usando uma variável global antes de inicializá-la, na primeira linha de nossa função. Essa definição é feita com a instrução global.

Variáveis locais e globais

```
a=5  
  
def muda_e_imprime():  
    global a ❶  
    a=7 ❷  
    print("A dentro da função: %d" % a)  
print("a antes de mudar: %d" % a)  
muda_e_imprime()  
print("a depois de mudar: %d" % a)
```

Agora, em ❶, estamos trabalhando com a variável a global. Assim, quando fizemos a=7 em ❷, trocamos o valor de a.

Funções recursivas

- Uma função pode chamar a si mesma.
- Quando isso ocorre temos uma função recursiva.

Funções recursivas

- Podemos definir o fatorial de um número como sendo esse número multiplicado pelo fatorial de seu antecessor.

$$fatorial(n) = \begin{cases} 1 & 0 \leq n \leq 1 \\ n \times fatorial(n - 1) & \end{cases}$$

Função recursiva do fatorial

```
def fatorial(n):  
    if n==0 or n == 1:  
        return 1  
    else:  
        return n*fatorial(n-1)
```

Função recursiva do fatorial

```
def fatorial(n):  
    print("Calculando o fatorial de %d" % n)  
    if n==0 or n == 1:  
        print("Fatorial de %d = 1" % n)  
        return 1  
    else:  
        fat = n*fatorial(n-1)  
        print(" fatorial de %d = %d" % (n, fat) )  
    return fat  
  
fatorial(4)
```

* Função modificada para facilitar o rastreamento

Tratamento de exceções

- O tratamento de exceções em Python é uma forma de lidar com erros que podem ocorrer durante a execução do programa, permitindo que você controle o fluxo do programa mesmo em situações inesperadas. O tratamento de exceções é feito usando o bloco try-except.

Tratamento de exceções

- Bloco "try": Você envolve o código que pode gerar uma exceção dentro do bloco try.

try:

Código que pode gerar exceções

...

except TipoDeExcecao:

Tratamento da exceção do tipo TipoDeExcecao

else:

Sem execucao

finally:

Sempre executado

Tratamento de exceções

- Bloco "except": Se ocorrer uma exceção dentro do bloco "try", o controle será transferido imediatamente para o bloco except, onde você pode tratar a exceção de acordo com o tipo dela.
- Tipo de exceção: Na cláusula except, você especifica o tipo de exceção que deseja capturar e tratar. Por exemplo, se você quiser capturar uma exceção de divisão por zero, usaria except ZeroDivisionError:.
- Tratamento da exceção: Dentro do bloco except, você coloca o código que será executado quando a exceção do tipo especificado ocorrer. Isso pode incluir a exibição de uma mensagem de erro, registrar informações, ou tomar qualquer ação necessária para lidar com a situação de erro.

Tratamento de exceções

try:

```
numero1 = int(input("Digite um número: "))
```

```
numero2 = int(input("Digite outro número: "))
```

```
resultado = numero1 / numero2
```

```
print("Resultado da divisão:", resultado)
```

except ZeroDivisionError:

```
print("Erro: Divisão por zero não é permitida.")
```

except ValueError:

```
print("Erro: Certifique-se de que você digitou números inteiros válidos.")
```

except Exception as e:

```
print("Ocorreu um erro inesperado:", e)
```

else:

```
print("Não ocorreram exceções")
```

finally:

```
print("Finalizando")
```


Tratamento de exceções

- Neste exemplo, o programa tenta obter dois números do usuário e realiza uma divisão entre eles. Se ocorrer uma exceção, o programa tratará especificamente os erros de divisão por zero ou valores inválidos. Caso uma exceção diferente seja lançada, o bloco `except Exception as e` capturará e tratará qualquer outra exceção, fornecendo uma mensagem genérica de erro.
- É importante notar que é uma boa prática capturar apenas as exceções que você pode tratar adequadamente e evitar capturar exceções genéricas em todo o código, pois isso pode dificultar a depuração de erros.

Classe

- Uma classe é um "modelo" ou "planta" que define a estrutura e o comportamento de objetos. Ela representa um tipo de dado personalizado, que pode conter atributos (variáveis) e métodos (funções) para manipular esses atributos. As classes são usadas para criar objetos, que são instâncias específicas da classe.

Classe

```
class NomeDaClasse:
    def __init__(self, parametro1, parametro2):
        # Construtor - define os atributos da classe
        self.atributo1 = parametro1
        self.atributo2 = parametro2

    def metodo1(self):
        # Método da classe
        # Pode acessar e manipular os atributos da classe usando 'self'
        # ...
```

Objetos

- Um objeto é uma instância específica de uma classe. Ele é criado a partir da classe e possui todas as características e comportamentos definidos pela classe.

Objetos

```
# Criação de um objeto da classe NomeDaClasse  
objeto1 = NomeDaClasse(valor1, valor2)
```

Objetos

- Atributos:
 - Os atributos são variáveis definidas dentro da classe e representam as características ou dados que o objeto possui. Cada objeto criado a partir da classe terá seus próprios valores para esses atributos.
- Métodos:
 - Os métodos são funções definidas dentro da classe que permitem ao objeto realizar ações e manipular seus atributos. Esses métodos são acessados através do objeto.

Objetos

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def saudacao(self):
        return f"Olá, meu nome é {self.nome} e tenho {self.idade} anos."

# Criando objetos da classe Pessoa
pessoa1 = Pessoa("João", 30)
pessoa2 = Pessoa("Maria", 25)

# Chamando o método saudacao() para cada objeto
print(pessoa1.saudacao()) # Saída: "Olá, meu nome é João e tenho 30 anos."
print(pessoa2.saudacao()) # Saída: "Olá, meu nome é Maria e tenho 25 anos."
```

Orientação a objetos

- Na orientação a objetos, você pode criar hierarquias de classes, onde uma classe pode herdar características e comportamentos de outra (herança). Além disso, é possível usar conceitos como encapsulamento, polimorfismo e abstração para organizar e estruturar o código de maneira mais eficiente e modular. Python oferece suporte a todos esses recursos de programação orientada a objetos, tornando-o uma linguagem poderosa para desenvolver sistemas complexos e reutilizáveis.

Hierarquia de classes

- Na programação orientada a objetos (POO), refere-se à organização de classes em uma estrutura hierárquica, onde as classes podem herdar características e comportamentos de outras classes. A classe que herda é chamada de "classe filha" ou "subclasse", e a classe da qual ela herda é chamada de "classe pai" ou "superclasse". Essa relação é também conhecida como relação "é um" (is-a), indicando que a subclasse é um tipo especializado da superclasse.

Hierarquia de classes

- Em Python, a hierarquia de classes é definida usando a sintaxe de herança. Para criar uma subclasse, você deve passar a classe pai como argumento na definição da subclasse. A subclasse herda todos os atributos e métodos da superclasse, e você pode adicionar novos atributos e métodos à subclasse, ou até mesmo sobrescrever os métodos herdados.

```
class Animal:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def emitir_som(self):
        return "Som indistinto"

class Cachorro(Animal): # Cachorro é uma subclasse de Animal
    def __init__(self, nome, idade, raca):
        # Chama o construtor da classe pai para definir nome e idade
        super().__init__(nome, idade)
        self.raca = raca

    def emitir_som(self):
        return "Au au!"

class Gato(Animal): # Gato é uma subclasse de Animal
    def __init__(self, nome, idade, cor):
        # Chama o construtor da classe pai para definir nome e idade
        super().__init__(nome, idade)
        self.cor = cor

    def emitir_som(self):
        return "Miau!"

# Criando objetos da classe Cachorro e Gato
cachorro1 = Cachorro("Rex", 3, "Labrador")
gato1 = Gato("Bola de Neve", 2, "Branco")

# Chamando métodos das subclasses
print(cachorro1.emitir_som()) # Saída: "Au au!"
print(gato1.emitir_som())     # Saída: "Miau!"

# Chamando métodos da superclasse
print(cachorro1.nome) # Saída: "Rex"
print(gato1.idade)    # Saída: 2
```

Hierarquia de classes

- Neste exemplo, temos uma classe Animal que é a superclasse, e duas subclasses: Cachorro e Gato. As subclasses herdam o construtor e o método emitir_som da superclasse, mas também têm atributos e métodos próprios. O método emitir_som é sobrescrito nas subclasses, dando a cada uma um comportamento específico.
- A hierarquia de classes é uma técnica poderosa para organizar o código, promovendo a reutilização e a extensibilidade. Ela permite que você crie classes especializadas com base em classes existentes, tornando o código mais eficiente e fácil de manter.

Encapsulamento

- Encapsulamento é a proteção dos atributos ou métodos de uma classe, em Python existem somente o public e o private e eles são definidos no próprio nome do atributo ou método.
- Atributos ou métodos iniciados por no máximo dois sublinhados e terminados por um sublinhado são privados e todas as outras formas são públicas.

Encapsulamento

```
class A:
    a = 1 # atributo publico
    __b = 2 # atributo privado a class A

class B(A):
    __c = 3 # atributo privado a B

    def __init__(self):
        print self.a
        print self.__c

a = A()
print a.a # imprime 1

b = B()
print b.__b # Erro, pois __b é privado a classe A.
print b.__c # Erro, __c é um atributo privado, somente chamado pela classe.

print b._B__c # Imprime __c = 3, muito pouco utilizada, mas existe.
```

Polimorfismo

- Deitel, em "Programando em Java", diz que "quando utilizamos herança para criar uma classe a partir da classe existente, a nova classe herda os atributos e comportamentos dessa classe existente, podendo adicionar atributos e comportamentos para solucionar nossas necessidades".
- O polimorfismo condiz com a reutilização de código: é fato que ocorrem funções semelhantes em várias partes do software; então definimos estas funções em uma biblioteca, e todas as outras funções que dela precisarem poderão chamá-la sem a necessidade de reescrevê-la.

Tipos de polimorfismo

- Polimorfismo de subtipagem (inclusão)
 - Objetos de subclasses são usados onde a superclasse é esperada. Ex.: uma função que aceita `Animal` pode receber `Cachorro` e chamar `emitir_som()` comportamento da subclasse é invocado.
- Polimorfismo paramétrico (genérico)
 - Código escrito sem dependência de tipos concretos; funciona para qualquer tipo que satisfaça requisitos (ex.: funções/genéricos em Python com `typing.TypeVar`, ou templates em C++). Permite reuso sem duplicação.

Tipos de polimorfismo

- Polimorfismo ad hoc
 - Sobrecarga de função/método: várias assinaturas com mesmo nome (presente em linguagens como C++, Java; Python não suporta diretamente).
 - Sobrecarga por coerção/conversão: operadores ou funções aceitam tipos diferentes através de conversões implícitas.
- Polimorfismo por sobrecarga de operadores
 - Objetos definem comportamento para operadores (ex.: add, len em Python), permitindo usar os mesmos operadores com diferentes tipos
- Polimorfismo de subtipagem por interface/duck typing
 - Em linguagens dinâmicas (Python), qualquer objeto que implemente métodos esperados pode ser usado — "se anda como pato e grasna como pato...". Não exige herança explícita neste caso.

Exceções

- Em Python, você pode criar exceções personalizadas criando suas próprias classes de exceção. As classes de exceção personalizadas herdam da classe `Exception` ou de suas subclasses para criar suas próprias exceções específicas. A criação de exceções personalizadas é útil quando você deseja lidar com erros específicos em seu código de uma maneira mais detalhada e informativa.

Exceções

```
class MeuErroPersonalizado(Exception):  
    def __init__(self, mensagem):  
        self.mensagem = mensagem  
  
    def __str__(self):  
        return f"Erro Personalizado: {self.mensagem}"  
  
def dividir(a, b):  
    if b == 0:  
        raise MeuErroPersonalizado("Divisão por zero não é permitida.")  
    return a / b  
  
try:  
    resultado = dividir(10, 0)  
except MeuErroPersonalizado as e:  
    print(e) # Saída: "Erro Personalizado: Divisão por zero não é permitida"
```

Exceções

- Neste exemplo, criamos uma classe `MeuErroPersonalizado` que herda de `Exception`. A classe tem um construtor `__init__` que permite definir a mensagem de erro personalizada, e um método `__str__` para retornar uma representação de string da exceção.
- A função `dividir()` é definida para realizar uma divisão, mas primeiro verifica se o divisor é zero. Se for zero, ela lança uma exceção `MeuErroPersonalizado`, informando que a divisão por zero não é permitida.

Exceções

- Em seguida, usamos um bloco try-except para tratar a exceção personalizada. Quando a exceção é lançada durante a execução da função dividir(), ela é capturada pelo bloco except MeuErroPersonalizado as e, onde e é a instância da exceção personalizada. O bloco except exibe a mensagem personalizada da exceção.

Exceções

- Com exceções personalizadas, você pode fornecer mensagens de erro mais significativas e personalizadas, facilitando a identificação e correção de problemas em seu código. Lembre-se de usar exceções personalizadas quando for apropriado e de capturar apenas as exceções que você pode lidar adequadamente. Evite capturar exceções genéricas em todo o código, pois isso pode dificultar a depuração de erros.

Extra

- Pense em Python 2/3ed
 - <https://penseallen.github.io/PensePython2e/>
 - <https://rodrigocarlson.github.io/PensePython3ed/>
- Documentação oficial do Python3
 - <https://docs.python.org/3/>
- Curso em vídeo: Python3
 - https://www.youtube.com/watch?v=S9uPNppGsGo&list=PLHz_AreHm4dIKP6QQCekuIPky1Ciwmdl6&pp=iAQB
 - https://www.youtube.com/watch?v=nJkVHusJp6E&list=PLHz_AreHm4dk_nZHmxxf_J0WRAqy5Czye&pp=iAQB
 - https://www.youtube.com/watch?v=0LB3FSfjvao&list=PLHz_AreHm4dksnH2jVTIVNviIMBVYyFnH&pp=iAQB